

CityPAJ

Memoria del Proyecto

Ciclo Formativo de Grado Superior en Desarrollo de Aplicaciones Web

Módulo de Proyecto

CPIFP Los Enlaces

Curso 2025/2026

Alumna: Carmen de Miguel Velázquez

Fecha de entrega: 16/08/2026

Índice general

- Documento Descripción del proyecto
- Documento de Acuerdo del proyecto
- Documento de Análisis y Diseño
- Documento de Implementación, Pruebas e Implantación del sistema
- Documento de Cierre
- Bibliografía
- Anexos
- Anexos técnicos adicionales

Índice de tablas

Las tablas del documento se numeran en orden de aparición. Entre las principales se encuentran: Tabla 1. Requisitos funcionales; Tabla 2. Requisitos no funcionales; Tabla 3. Historias de usuario; Tabla 4. Tareas; Tabla 5. Tecnologías; Tabla 6. Modelo relacional; Tabla 7. Plan de pruebas; Tabla 8. Anexos.

Índice de ilustraciones

Figura 1. Arquitectura general; Figura 2. Flujo usuario-frontend-backend; Figura 3. Modelo entidad-relación; Figura 4. Modelo relacional; Figura 5. Casos de uso; Figura 6. Secuencia de login; Figura 7. Secuencia de publicación de anuncio; Figura 8. Estados de anuncio; Figura 9. Arquitectura Docker; Figura 10. Módulos frontend; Figura 11. Módulos backend; Figura 12. Flujo de moderación; Figura 13. Flujo de sugerencias y propuestas; Figura 14. Flujo de comunidad.

Índice general

Índice de tablas

Índice de ilustraciones

Documento Descripción del proyecto

1.1. Contexto del proyecto

CityPAJ nace dentro del módulo de Proyecto del ciclo de Desarrollo de Aplicaciones Web del CPIFP Los Enlaces. El proyecto responde a la necesidad de construir una aplicación web completa que sirva de demostración de las competencias adquiridas durante el ciclo.

El entorno actual de la información juvenil presenta una alta dispersión: los jóvenes consultan anuncios, convocatorias, ofertas de empleo, recursos culturales y avisos comunitarios en múltiples fuentes. Esta fragmentación dificulta la búsqueda eficiente y la participación activa.

La propuesta de CityPAJ consiste en una plataforma centralizada en la que la información se organiza por territorio, categoría y tipo de contenido. El sistema permite la publicación, consulta y moderación de anuncios, propuestas, sugerencias y recursos, con acceso diferenciado según el rol de usuario.

1.1.1. Ámbito y entorno

El ámbito del proyecto es el desarrollo de software para entornos web. consiste en una aplicación de uso local mediante Docker Compose, pensada para ser revisada por el tribunal académico sin depender de despliegues externos.

El entorno tecnológico elegido refleja las tecnologías trabajadas en el ciclo: frontend con Next.js, React y TypeScript; backend con Node.js y Express; y base de datos relacional con MySQL. La dockerización asegura que cualquier revisador pueda levantar el sistema con un solo comando.

La aplicación no requiere conexión a servicios de terceros para funcionar de forma básica. Todas las dependencias principales están contenidas en el repositorio o descargables mediante npm y Docker Hub.

1.1.2. Análisis de la realidad

La realidad que motiva el proyecto es la dificultad de acceso a la información juvenil relevante por territorio. En la práctica, un usuario interesado en empleo, formación o vivienda juvenil debe consultar portales institucionales, redes sociales, grupos de mensajería o tableros físicos, lo que genera pérdida de oportunidades.

adicionalmente, la separación entre anuncios, propuestas, sugerencias, cultura y recursos no siempre es clara en los portales existentes. CityPAJ aborda esta confusión definiendo secciones con propósito diferenciado y filtros por comunidad autónoma y provincia.

La participación juvenil asimismo se observa limitada cuando los canales no permiten aportar propuestas o sugerencias de forma sencilla. El proyecto incluye un buzón de sugerencias y un apartado de propuestas para fomentar esa participación.

1.1.3. Solución y justificación de la solución propuesta

La solución propuesta es una aplicación web fullstack con arquitectura en tres capas. El frontend consume la API REST del backend, que a su vez persiste la información en una base de datos MySQL.

La organización por comunidades autónomas y provincias permite que el usuario consulte solo la información territorialmente relevante. Los filtros por categoría permiten separar anuncios generales de cultura, eventos, recursos y propuestas.

La justificación técnica recae en el dominio de tecnologías estudiadas y en la facilidad de entrega. Docker Compose reduce los problemas de configuración local y hace el proyecto más reproducible. La separación frontend/backend facilita el mantenimiento y la evolución futura.

1.1.4. Destinatarios

Los destinatarios principales son los jóvenes que buscan oportunidades de empleo, formación, vivienda, ocio o servicios en su comunidad.

Entidades e instituciones juveniles asimismo son destinatarias, puesto que pueden publicar recursos, convocatorias o actividades y mantener contacto con el público objetivo.

Los moderadores son responsables de supervisar los contenidos publicados y resolver reportes de los usuarios.

Los administradores gestionan usuarios, categorías, tareas, comunicaciones y acceso al sistema.

1.2. Objetivo del proyecto

El objetivo del proyecto es desarrollar una aplicación web que permita centralizar información juvenil organizada por territorio, facilitando la consulta y publicación de anuncios, recursos, propuestas, sugerencias y actividad comunitaria.

El sistema se dirige a jóvenes, entidades y administraciones que necesitan un punto de acceso unificado a información juvenil. El problema que resuelve es la dispersión de anuncios y recursos en múltiples canales sin posibilidad de filtrado territorial.

Entre los módulos incluidos se encuentran autenticación de usuarios, publicación de anuncios con moderación, comunidad por provincias, buzón de sugerencias, propuestas, cultura y eventos, panel de administración, panel de moderación y mensajería interna.

A nivel técnico, el proyecto aporta una arquitectura separada en frontend y backend, tipado estático con TypeScript, validación de datos, control de roles y dockerización local.

A nivel social, el proyecto aporta una herramienta orientada a mejorar el acceso y la participación juvenil, con controles de moderación para mantener la calidad de la información.

1.3. Project Objective

The aim of this project is to develop a web application that gathers youth information organised by territory. It allows users to consult and publish announcements, resources, proposals, suggestions and community activities.

The system is divided into a Next.js frontend, an Express backend and a MySQL database. It supports user roles, content moderation and a local Docker deployment.

The project offers a useful tool for young people, organisations and administrations to centralise youth information and improve participation.

1.4. Marco legal

El proyecto maneja datos personales mínimos, como dirección de correo electrónico, contraseña cifrada e, en algunos casos, dirección IP. Si el sistema se desplegara en un entorno real, debería cumplir el Reglamento General de Protección de Datos (RGPD) y la Ley Orgánica 3/2018 de Protección de Datos Personales y garantía de los derechos digitales.

La contraseña se almacena cifrada con bcrypt, lo que reduce el riesgo de exposición en caso de acceso no autorizado a la base de datos. El uso de tokens JWT permite gestionar sesiones sin enviar la contraseña en cada petición.

El sistema separa roles de usuario, de forma que solo los administradores y moderadores acceden a funciones de gestión. La moderación de contenidos constituye una medida de control de calidad y legalidad.

La licencia declarada en el archivo package.json del repositorio es MIT. Sin embargo, el proyecto se entrega con finalidad académica para su revisión dentro del módulo de Proyecto. La reutilización o publicación fuera de este contexto queda sujeta a la autorización de la autora.

Documento de Acuerdo del proyecto

2.1. Requisitos funcionales

A continuación se listan los requisitos funcionales identificados en el repositorio. Cada requisito incluye el actor principal, el estado observado y una prueba asociada.

ID

ID	Requisito funcional	Descripción	Actor principal	Estado	Prueba asociada
RF-01	Registro de usuarios	Permite crear una cuenta con correo, nombre y contraseña.	Visitante	Cumplido	POST /api/auth/register
RF-02	Inicio de sesión	Permite autenticarse mediante correo y contraseña.	Usuario	Cumplido	POST /api/auth/login
RF-03	Cierre de sesión	Permite cerrar la sesión activa.	Usuario	Cumplido	POST /api/auth/logout
RF-04	Gestión de sesión	Renovación y control del token JWT.	Usuario	Cumplido	POST /api/auth/refresh
RF-05	Consulta de anuncios	Listado paginado y búsqueda de anuncios.	Visitante	Cumplido	Carga del home

RF-06	Filtrado por categoría	Filtro por tipo como empleo, formación, vivienda, cultura.	Usuario	Cumplido	GET /api/anuncios?categoria=
RF-07	Filtrado por territorio	Filtro por comunidad autónoma y provincia.	Usuario	Cumplido	GET /api/anuncios?comunidad=&provincia=
RF-08	Publicación de anuncios	Formulario de publicación con adjuntos.	Usuario	Cumplido	POST /api/anuncios
RF-09	Edición de anuncios propios	Permite modificar anuncios del usuario.	Usuario	Parcial	PUT /api/anuncios/:id
RF-10	Detalle de anuncio	Visualización completa de un anuncio.	Usuario	Cumplido	GET /api/anuncios/:id
RF-11	Gestión de favoritos	Guardar y consultar anuncios favoritos.	Usuario	Cumplido	POST /api/anuncios/guardar
RF-12	Reporte de anuncios	Enviar reporte por contenido inadecuado.	Usuario	Cumplido	POST /api/anuncios/:id/reportes
RF-13	Comunidad por provincias	Publicaciones locales y comentarios.	Usuario	Parcial	GET /api/comunidad
RF-14	Publicación en comunidad	Permite crear publicaciones territoriales.	Usuario	Parcial	POST /api/comunidad
RF-15	Envío de sugerencias	Buzón para sugerencias juveniles.	Usuario	Cumplido	POST /api/sugerencias
RF-16	Consulta de propuestas	Visualización de propuestas ciudadanas.	Usuario	Cumplido	GET /api/propuestas
RF-17	Apoyo a propuestas	Registro de apoyos a propuestas.	Usuario	Parcial	POST /api/propuestas/:id/apoyos
RF-18	Consulta de recursos	Listado de recursos juveniles.	Usuario	Parcial	GET /api/recursos
RF-19	Consulta de cultura y eventos	Eventos culturales y	Usuario	Cumplido	GET /api/eventos

		actividades.			
RF-20	Panel de administración	Gestión de usuarios, anuncios, tareas y estadísticas.	Admin	Parcial	GET /api/admin
RF-21	Gestión de usuarios	Alta, baja y modificación de usuarios.	Admin	Parcial	CRUD /api/admin/usuarios
RF-22	Gestión de anuncios	Revisión masiva y edición de anuncios.	Admin	Parcial	GET/POST /api/admin/anuncios
RF-23	Panel de moderación	Revisión de reportes y publicaciones pendientes.	Moderador	Parcial	GET /api/moderacion
RF-24	Revisión de reportes	Acciones sobre reportes de contenido.	Moderador	Parcial	GET /api/moderacion/reportes
RF-25	Mensajería interna	Envío de mensajes entre usuarios del staff.	Admin/Moderador	Parcial	GET/POST /api/admin/mensajes
RF-26	Agenda institucional	Notas y fechas para administración.	Admin	Parcial	GET /api/admin/agenda
RF-27	Comunicaciones institucionales	Borradores de comunicaciones con entidades.	Admin	Parcial	GET /api/admin/comunicaciones
RF-28	Plantillas de comunicación	Gestión de plantillas para mensajes.	Admin	Parcial	GET /api/admin/plantillas
RF-29	Dockerización y ejecución local	Levantar la aplicación con Docker Compose.	Sistema	Cumplido	docker compose up --build
RF-30	Health check del backend	Endpoint de comprobación de salud.	Sistema	Cumplido	GET /health

2.2. Requisitos no funcionales

Los requisitos no funcionales definen las cualidades que debe cumplir el sistema.

ID

ID	Requisito	Descripción	Justificación	Estado
----	-----------	-------------	---------------	--------

RNF-01	Usabilidad	Interfaz clara y navegación por secciones.	Reduce curva de aprendizaje.	Cumplido
RNF-02	Accesibilidad básica	Contraste, etiquetas y estructura semántica.	Mejora legibilidad.	Parcial
RNF-03	Diseño responsive	Adaptación a móvil y escritorio.	Uso en distintos dispositivos.	Cumplido
RNF-04	Seguridad en autenticación	Contraseñas cifradas con bcrypt y JWT.	Protege cuentas.	Cumplido
RNF-05	Validación de datos	Esquemas con Zod y validaciones de formulario.	Reduce datos erróneos.	Cumplido
RNF-06	Mantenibilidad	Código modular en capas.	Facilita correcciones.	Cumplido
RNF-07	Separación frontend/backend	Arquitectura cliente-servidor.	Claridad y escalabilidad.	Cumplido
RNF-08	Persistencia en MySQL	Base de datos relacional con esquema.	Almacenamiento estructurado.	Cumplido
RNF-09	Portabilidad con Docker	Contenedores para toda la pila.	Ejecución sin instalaciones.	Cumplido
RNF-10	Rendimiento suficiente	Respuestas aceptables para entorno académico.	Uso en tribunal.	Parcial
RNF-11	Compatibilidad	Navegadores modernos.	Uso generalizado.	Cumplido
RNF-12	Claridad visual	Diseño con Tailwind CSS.	Consistencia visual.	Cumplido
RNF-13	Protección básica de datos	Tratamiento mínimo y roles.	Cumplimiento básico.	Cumplido
RNF-14	Escalabilidad futura	Arquitectura modular.	Permite ampliaciones.	Parcial
RNF-15	Documentación de instalación	README y memoria.	Facilita la entrega.	Cumplido
RNF-16	Control de errores	Manejo centralizado de errores.	Mejora robustez.	Parcial
RNF-17	Organización del código	Carpetas por responsabilidad.	Mantenimiento.	Cumplido
RNF-18	Reutilización de componentes	Componentes reutilizables en frontend.	Eficiencia.	Parcial

RNF-19	Consistencia de interfaz	Paleta y componentes comunes.	Experiencia unificada.	Cumplido
RNF-20	Facilidad de revisión	Docker y README claros.	Evaluación del tribunal.	Cumplido

2.3. Historias de usuario

Las historias de usuario definen las interacciones de cada perfil con el sistema.

ID

ID	Historia de usuario	Criterios de aceptación	Requisitos
HU-01	Como visitante, quiero ver anuncios, para conocer oportunidades juveniles.	El home muestra listado de anuncios.	RF-05
HU-02	Como visitante, quiero filtrar por categoría, para encontrar contenido relevante.	Filtros de categoría funcionan.	RF-06
HU-03	Como usuario, quiero registrarme, para publicar contenido.	Registro con datos válidos crea cuenta.	RF-01
HU-04	Como usuario, quiero iniciar sesión, para acceder a mi perfil.	Login devuelve token y redirige.	RF-02
HU-05	Como usuario, quiero publicar un anuncio, para compartir una oportunidad.	El formulario valida y guarda.	RF-08
HU-06	Como usuario, quiero filtrar por provincia, para ver anuncios cercanos.	Selector de provincia aplica filtro.	RF-07
HU-07	Como usuario, quiero guardar favoritos, para consultarlos después.	El anuncio aparece en guardados.	RF-11
HU-08	Como usuario, quiero reportar contenido, para mantener la calidad.	Se envía reporte con motivo.	RF-12
HU-09	Como usuario, quiero enviar sugerencias, para proponer mejoras.	El buzón almacena la sugerencia.	RF-15
HU-10	Como usuario, quiero consultar propuestas, para apoyar ideas.	Página de propuestas accesible.	RF-16
HU-11	Como usuario, quiero consultar cultura y eventos, para conocer actividades.	Sección cultura funcional.	RF-19

HU-12	Como moderador, quiero revisar anuncios pendientes, para aprobar o rechazar.	Panel muestra pendientes.	RF-23
HU-13	Como moderador, quiero gestionar reportes, para resolver incidencias.	Listado de reportes con acciones.	RF-24
HU-14	Como administrador, quiero gestionar usuarios, para controlar el acceso.	CRUD de usuarios.	RF-21
HU-15	Como administrador, quiero ver estadísticas, para entender el uso.	Dashboard con métricas.	RF-20

2.4. Definición de tareas

El proyecto se descompone en tareas organizadas por fases.

ID

ID	Fase	Tarea	Descripción	Estimadas	Reales	Resultado
T-01	Análisis inicial	Definir alcance y requisitos	Estudio de la idea y funcionalidades básicas.	12	15	Completado
T-02	Diseño funcional	Diseñar casos de uso	Identificación de actores y flujos.	10	10	Completado
T-03	Diseño BD	Modelar entidades	Esquema relacional en MySQL.	12	14	Completado
T-04	Diseño UI	Prototipar pantallas	Bocetos de páginas principales.	10	8	Completado
T-05	Configuración	Preparar repositorio	Inicialización y estructura de carpetas.	6	8	Completado
T-06	Frontend	Desarrollar páginas	Implementación de rutas y componentes.	40	48	Completado
T-07	Backend	Desarrollar API	Controladores, rutas y middleware.	40	45	Completado
T-08	Base de datos	Conectar y poblar	Conexión mysql2 y datos demo.	10	12	Completado
T-09	Autenticación	Login y roles	Registro, login y tokens	10	12	Completado

			JWT.			
T-10	Anuncios	Publicación y consulta	CRUD y filtros de anuncios.	18	22	Completado
T-11	Filtros	Filtros territoriales	Comunidades y provincias.	8	10	Completado
T-12	Comunidad	Publicaciones y comentarios	Foros por provincia.	12	10	Parcial
T-13	Sugerencias	Buzón	Envío y listado de sugerencias.	6	6	Completado
T-14	Propuestas	Propuestas y apoyos	Visualización y apoyo.	8	7	Parcial
T-15	Cultura y eventos	Sección cultural	Eventos y actividades.	10	10	Completado
T-16	Administración	Panel admin	Módulos de gestión.	20	24	Parcial
T-17	Moderación	Panel moderador	Revisión y reportes.	12	14	Parcial
T-18	Mensajería	Mensajes staff	Vista de mensajes.	10	8	Parcial
T-19	Docker	Dockerización	Contenedores y Compose.	8	12	Completado
T-20	Pruebas	Pruebas funcionales	Validación manual y con usuarios demo.	10	6	Parcial
T-21	Documentación	README y memoria	Redacción de documentos.	12	14	Completado
T-22	Cierre	Revisión final	Limpieza y entrega.	6	8	En curso

2.5. Metodología

El proyecto utiliza una metodología mixta, puesto que combina una fase inicial de análisis y diseño con una implementación incremental. Esta forma de trabajo resulta adecuada para un proyecto académico, dado que permite definir primero una base funcional y después ampliar o corregir módulos según las necesidades detectadas durante el desarrollo.

La fase de análisis inicial permitió identificar los requisitos básicos y la arquitectura. La fase de diseño definió el esquema de base de datos, las pantallas principales y la estructura de carpetas.

El desarrollo incremental se organizó en bloques: autenticación, anuncios, filtros, comunidad, sugerencias, propuestas, cultura, administración, moderación y Docker. Cada bloque se probó de forma manual antes de pasar al siguiente.

La revisión por entregas consistió en realizar commits con funcionalidades concretas. El historial de Git refleja esta evolución, con mensajes que describen el trabajo realizado en cada iteración.

La corrección de errores se realizó de forma progresiva. Cuando un módulo presentaba problemas, se detenía el avance, se depuraba y se continuaba. Este enfoque evitó acumular fallos de difícil resolución.

La documentación final se redactó una vez estabilizada la aplicación. El README contiene las instrucciones de instalación y la memoria describe el proyecto completo.

2.6. Planificación temporal

La planificación se organiza en fases con horas previstas y reales.

Fase

Fase	Previsto	Real	Desviación	Motivo
Análisis inicial	12	15	+3	Ajustes de alcance.
Diseño	22	24	+2	Refinamiento de BD y UI.
Configuración	6	8	+2	Problemas iniciales con Docker.
Frontend	40	48	+8	Ajustes de componentes.
Backend	40	45	+5	Validaciones y tipado.
Base de datos	10	12	+2	Inserción de datos demo.
Docker	8	12	+4	Configuración del proxy.
Pruebas	10	6	-4	Tiempo reducido por documentación.
Documentación	12	14	+2	Ampliación del README y memoria.

La desviación más importante se produjo en el frontend y en la dockerización. El ajuste del proxy entre frontend y backend en Docker requirió varias iteraciones. La planificación final se ha adaptado a estos retrasos sin perder los objetivos principales.

2.7. Presupuesto

El presupuesto es académico y no contempla costes reales de contratación. Las horas corresponden al trabajo de la alumna y los costes de software son cero al emplear herramientas libres o gratuitas.

Concepto

Concepto	Horas	Coste unitario	Total	Observación
Análisis	15	0	0	Trabajo propio
Diseño funcional	12	0	0	Trabajo propio

Diseño BD	14	0	0	Trabajo propio
Diseño UI	8	0	0	Trabajo propio
Implementación frontend	48	0	0	Trabajo propio
Implementación backend	45	0	0	Trabajo propio
Integración BD	12	0	0	Trabajo propio
Pruebas	6	0	0	Trabajo propio
Docker	12	0	0	Trabajo propio
Documentación	14	0	0	Trabajo propio
Memoria	10	0	0	Trabajo propio
Herramientas software	-	-	0	Next.js, Node, MySQL, Docker libres
Despliegue local	-	-	0	Docker Desktop

Las herramientas utilizadas son de libre disposición: Node.js, npm, Next.js, React, TypeScript, Express, MySQL, Docker y Docker Compose. No se han contratado servicios externos ni licencias de pago. El coste total del presupuesto para el entorno académico es, por tanto, cero.

2.8. Licencia o condiciones de distribución

El archivo package.json del repositorio declara una licencia MIT. Sin embargo, el proyecto se entrega con finalidad académica para su revisión dentro del módulo de Proyecto.

La reutilización o publicación fuera de este contexto queda sujeta a la autorización de la autora. El tribunal puede consultar el código y ejecutar la aplicación para evaluación, pero no está autorizado a redistribuir el proyecto sin consentimiento.

2.9. Análisis de riesgos

El análisis de riesgos identifica posibles problemas y las medidas para mitigarlos.

ID

ID	Riesgo	Probabilidad	Impacto	Medida preventiva	Medida correctiva
R-01	Retraso en implementación	Media	Alta	Planificación con márgenes.	Reducir alcance si es necesario.
R-02	Errores de conexión con MySQL	Media	Media	Uso de Docker y healthchecks.	Revisar variables de entorno.
R-03	Problemas con Docker	Media	Media	Pruebas previas.	Documentar soluciones en README.
R-04	Problemas de autenticación	Baja	Alta	Pruebas con usuarios demo.	Revisar tokens y roles.

R-05	Pérdida de datos	Baja	Media	Copias en SQL y Git.	Restaurar desde init.sql.
R-06	Validaciones incompletas	Media	Media	Uso de Zod y express-validator.	Añadir validaciones.
R-07	Problemas responsive	Baja	Media	Diseño mobile-first.	Ajustar estilos.
R-08	Incompatibilidad de versiones	Baja	Baja	Fijar versiones.	Actualizar package.json.
R-09	Errores en despliegue local	Media	Media	Docker Compose documentado.	Revisar logs.
R-10	Alcance excesivo	Media	Alta	Priorizar módulos.	Marcar funciones como parciales.
R-11	Falta de tiempo para documentación	Media	Media	Documentar durante el desarrollo.	Priorizar secciones.
R-12	Errores en rutas API	Baja	Alta	Pruebas de endpoints.	Revisar controladores.
R-13	Errores en permisos de usuario	Baja	Alta	Middleware de roles.	Revisar protecciones.
R-14	Consistencia territorial	Media	Media	Datos de comunidades y provincias.	Corregir seeds.

Documento de Análisis y Diseño

3.1. Análisis y diseño de la arquitectura de la aplicación

La arquitectura de CityPAJ sigue un patrón de tres capas correctamente diferenciadas: presentación, lógica de negocio y persistencia. Esta separación permite distribuir responsabilidades y facilita la revisión del código.

La capa de presentación está implementada con Next.js 14. Next.js proporciona enrutamiento basado en archivos, renderizado del lado del servidor y la posibilidad de crear API internas de proxy. El frontend consume el backend principal a través de rutas API de Next.js para evitar problemas de CORS en entorno local.

La capa de lógica de negocio está implementada con Express. Los controladores reciben las peticiones, aplican validaciones, consultan la base de datos y devuelven respuestas JSON. Los middlewares gestionan autenticación, rate limiting y CORS.

La capa de persistencia utiliza MySQL 8.0. El esquema se carga mediante `database/init.sql` al iniciar el contenedor. La conexión se realiza con el driver `mysql2` y un pool de conexiones para mejorar el rendimiento.

La comunicación sigue el flujo: usuario -> navegador -> frontend (puerto 3001) -> backend (puerto 3002) -> MySQL (puerto 3306). Dentro de Docker, el frontend se comunica con el backend a través de la URL interna `http://backend:3002`, mientras que el navegador usa `http://localhost:3002`.

La arquitectura lógica se apoya en una API REST. Los endpoints responden a los métodos HTTP GET, POST, PUT, DELETE y PATCH. Cada recurso tiene un controlador asociado que separa la lógica de las rutas.

3.2. Tecnologías y herramientas utilizadas

Las tecnologías se han seleccionado en función de las competencias trabajadas en el ciclo y de la disponibilidad en el entorno académico.

Tecnología

Tecnología	Versión	Uso	Justificación
Next.js	14.x	Framework frontend	SSR, enrutamiento y componentes React.
React	18.x	Biblioteca de interfaz	Componentes reutilizables.
TypeScript	5.2.x	Tipado estático	Reduce errores.
Tailwind CSS	3.3.x	Estilos	Maquetación rápida y consistente.
Node.js	>=20	Entorno backend	Ejecución de Express.
Express	4.18.x	API REST	Middlewares y controladores.
MySQL	8.0	Base de datos	Persistencia relacional.
mysql2	3.22.x	Driver MySQL	Conexión y consultas.
bcryptjs	2.4.x	Hash de contraseñas	Seguridad de autenticación.
jsonwebtoken	9.0.x	JWT	Gestión de sesiones.
Zod	3.22.x	Validación	Esquemas de formularios.
Docker	26.x compatible	Contenedores	Ejecución local.
Docker Compose	2.x	Orquestación	Levantar servicios.
Git	2.x	Control de versiones	Historial y ramas.
GitHub	-	Repositorio remoto	Almacenamiento y revisión.
npm	9.x	Gestor de paquetes	Dependencias.
multer	1.4.x	Subida de archivos	Carteles e imágenes.
axios	1.5.x	Peticiones HTTP	Comunicación frontend.
lucide-react	0.562.x	Iconos	Interfaz visual.
helmet	7.0.x	Seguridad HTTP	Cabeceras de seguridad.
express-rate-limit	7.1.x	Rate limiting	Protección de rutas.

3.3. Arquitectura de componentes

La arquitectura de componentes se divide en tres capas principales, cada una con responsabilidades claras.

Capa

Capa	Tecnología	Responsabilidad
Presentación	Next.js y React	Renderizar páginas, manejar navegación y capturar eventos de usuario.

Aplicación	Express	Definir endpoints, autenticación, validación y lógica de negocio.
Datos	MySQL	Almacenar usuarios, anuncios, comunidad, sugerencias y resto de entidades.
Infraestructura	Docker	Levantar servicios y configurar red interna.

El frontend se organiza en carpetas app, components, hooks, context y lib. La carpeta app contiene las páginas de Next.js. Los componentes se dividen por área: anuncios, comunidad, admin, ui y layout. Los hooks encapsulan el acceso a datos y los contextos gestionan el estado global, como la autenticación.

El backend se organiza en controllers, routes, middleware, config, utils, services y types. Los controladores implementan la lógica de cada recurso. Las rutas exponen los endpoints. Los middlewares protegen rutas y aplican validaciones. La configuración centraliza variables de entorno y conexión a base de datos.

La separación de responsabilidades permite modificar el frontend sin afectar al backend y viceversa. asimismo facilita la realización de pruebas unitarias de cada capa.

3.4. Modelado de datos

3.4.1. Datos de entrada

Los datos de entrada del sistema provienen de las interacciones de los usuarios con los formularios y los paneles. Se pueden clasificar en varios grupos.

Datos de registro: correo electrónico, nombre y contraseña. Estos datos se validan en frontend y backend antes de almacenarse.

Datos de anuncios: título, descripción, categoría, subcategoría, comunidad autónoma, provincia, modalidad, contacto, precio y cartel. El sistema permite adjuntar una imagen de cartel.

Datos de comunidad: título, contenido, provincia, tema e IP del creador. La publicación puede ser anónima o con nombre.

Datos de sugerencias: asunto, descripción, categoría y prioridad. No requieren usuario registrado.

Datos de propuestas: título, descripción, comunidad autónoma y contenido. Los usuarios pueden apoyar una propuesta.

Datos de administración: tareas, agenda, comunicaciones, plantillas y logs de actividad. Estos datos se gestionan desde el panel admin.

Datos de autenticación: credenciales de inicio de sesión. El backend verifica la contraseña y emite un token JWT.

3.4.2. Datos de salida

Los datos de salida son las respuestas que el sistema devuelve a los usuarios. Incluyen páginas HTML generadas por Next.js y respuestas JSON de la API.

Respuestas de anuncios: listados paginados, detalles completos, resultados de búsqueda y filtros aplicados.

Respuestas de autenticación: token JWT, datos del usuario y roles.

Respuestas de administración: tablas de usuarios, anuncios, reportes, tareas y estadísticas.

Respuestas de error: mensajes descriptivos con códigos HTTP adecuados y, en algunos casos, información de depuración.

Respuestas de salud: el endpoint /health devuelve el estado de conexión con MySQL, la base de datos activa, el tiempo de actividad y la versión.

3.4.3. Datos almacenados

La base de datos citypaj almacena toda la información del sistema. El esquema se define en el archivo database/init.sql.

Entre las entidades principales se encuentran usuarios, anuncios, comunidad_publicaciones, comunidad_comentarios, comunidad_likes, sugerencias, propuestas, propuestas_apoyos, recursos, eventos, reportes_anuncios, anuncios_guardados, mensajes_staff, admin_tareas, agenda_notas, plantillas_comunicacion, comunicaciones_institucionales y admin_activity_logs.

asimismo se almacenan catálogos de comunidades y provincias para el filtrado territorial.

Los datos de demostración incluyen usuarios con roles admin, moderador y usuario, junto con anuncios de prueba, eventos culturales y publicaciones de comunidad.

3.4.4. Modelo entidad-relación

El modelo entidad-relación se construye a partir de las tablas presentes en database/init.sql.

La entidad usuarios es central. De ella dependen anuncios, comunidad_publicaciones, comunidad_comentarios, anuncios_guardados, reportes_anuncios, propuestas, mensajes_staff, admin_tareas y admin_activity_logs.

La entidad anuncios se relaciona con anuncios_guardados, reportes_anuncios y comunidad_publicaciones de forma indirecta a través de filtros y categorías.

La entidad comunidad_publicaciones se relaciona con comunidad_comentarios y comunidad_likes.

La entidad propuestas se relaciona con propuestas_apoyos.

Las entidades comunidades y provincias actúan como catálogos territoriales vinculados a anuncios, propuestas, eventos y comunidad.

El diagrama correspondiente se incluye en el anexo como Figura 3.

3.4.5. Modelo relacional

A continuación se presenta una tabla resumen de las tablas principales, sus campos y relaciones lógicas.

Tabla

Tabla	Finalidad	Campos principales	Relaciones
usuarios	Cuentas de usuario y roles	id, email, password_hash, nombre, rol, creado, actualizado	anuncios, comunidad, mensajes

anuncios	Publicaciones principales	id, usuario_id, titulo, descripcion, categoria, comunidad_id, provincia_id, estado_moderacion	favoritos, reportes
anuncios_guardados	Favoritos de usuarios	id, usuario_id, anuncio_id	usuarios, anuncios
comunidad_publicaciones	Publicaciones locales	id, usuario_id, titulo, contenido, provincia, tema, estado_moderacion	comentarios, likes
comunidad_comentarios	Comentarios en publicaciones	id, publicacion_id, usuario_id, contenido, visible	publicaciones
comunidad_likes	Me gusta en publicaciones	id, tipo, objeto_id, ip	publicaciones/comentarios
sugerencias	Sugerencias de usuarios	id, asunto, descripcion, categoria, prioridad	-
propuestas	Propuestas ciudadanas	id, usuario_id, titulo, descripcion, comunidad_id	propuestas_apoyos
propuestas_apoyos	Apoyos a propuestas	id, propuesta_id, usuario_id	propuestas
recursos	Recursos juveniles	id, titulo, descripcion, categoria, url	-
eventos	Eventos culturales	id, titulo, descripcion, fecha, provincia	-
reportes_anuncios	Reportes de anuncios	id, anuncio_id, motivo, usuario_id	anuncios
mensajes_staff	Mensajes internos	id, remitente_id, destinatario_id, asunto, cuerpo	usuarios
admin_tareas	Tareas del panel admin	id, titulo, descripcion, estado, asignado_a	usuarios
agenda_notas	Notas de agenda	id, titulo, cuerpo, fecha, usuario_id	usuarios
plantillas_comunicacion	Plantillas de mensajes	id, nombre, tipo, asunto, cuerpo	comunicaciones
comunicaciones_institucionales	Comunicaciones a entidades	id, plantilla_id, asunto, cuerpo, estado	plantillas
admin_activity_logs	Logs de actividad	id, usuario_id, accion, entidad, detalles, ip	usuarios
comunidades	Catálogo de CCAA	id, nombre	provincias
provincias	Catálogo de provincias	id, comunidad_id, nombre	comunidades

3.4.6. Script de base de datos

El script database/init.sql contiene la definición del esquema y los datos iniciales. Al arrancar el contenedor mysql, el volumen monta init.sql en /docker-entrypoint-initdb.d/, ejecutándose automáticamente.

El script realiza las siguientes operaciones: selecciona la base de datos citypaj, elimina las tablas si existen, crea la estructura con tipos de datos, claves primarias e índices, inserta datos de comunidades, provincias, usuarios, anuncios, publicaciones, comentarios, sugerencias, propuestas, recursos, eventos y logs de demostración.

El esquema utiliza claves foráneas en algunas tablas, como comunidad_comentarios hacia comunidad_publicaciones y usuarios. Otras relaciones son lógicas, gestionadas desde los controladores.

El script no debe ejecutarse manualmente salvo para reiniciar la base de datos. Para ello se recomienda docker compose down -v seguido de docker compose up --build.

3.5. Análisis y diseño del sistema funcional

3.5.1. Actores del sistema

Los actores son los perfiles que interactúan con el sistema.

Actor

Actor	Descripción	Funciones principales
Visitante	Usuario no autenticado	Consultar anuncios, recursos, cultura y eventos; buscar por categoría y territorio.
Usuario registrado	Cuenta con rol usuario	Publicar anuncios, guardar favoritos, enviar sugerencias, comentar en comunidad.
Moderador	Cuenta con rol moderador	Revisar anuncios pendientes, gestionar reportes, moderar contenido.
Administrador	Cuenta con rol admin	Gestionar usuarios, anuncios, tareas, comunicaciones, agenda y estadísticas.

3.5.2. Casos de uso

Los casos de uso describen las acciones principales que cada actor puede realizar.

ID

ID	Caso de uso	Actor	Descripción	Resultado esperado
CU-01	Registro	Visitante	Crear cuenta con email y contraseña.	Cuenta almacenada.
CU-02	Login	Usuario	Iniciar sesión.	Token JWT y redirección.
CU-03	Publicar anuncio	Usuario	Rellenar formulario	Anuncio en estado

			de anuncio.	pending.
CU-04	Buscar anuncios	Visitante	Aplicar filtros.	Listado filtrado.
CU-05	Guardar favorito	Usuario	Pulsar en favorito.	Anuncio en guardados.
CU-06	Reportar anuncio	Usuario	Enviar motivo.	Reporte registrado.
CU-07	Enviar sugerencia	Usuario	Rellenar buzón.	Sugerencia registrada.
CU-08	Apoyar propuesta	Usuario	Pulsar apoyo.	Registro de apoyo.
CU-09	Moderar anuncio	Moderador	Revisar y decidir.	Anuncio aprobado o rechazado.
CU-10	Gestionar usuarios	Administrador	Crear, editar o eliminar.	Usuario actualizado.
CU-11	Crear tarea	Administrador	Añadir tarea a otro usuario.	Tarea registrada.
CU-12	Enviar comunicación	Administrador	Usar plantilla.	Comunicación en borrador.

3.5.3. Flujo general de la aplicación

El flujo general es el siguiente. El usuario accede al frontend a través del navegador. Si la acción requiere datos, el frontend realiza una petición al backend. El backend recibe la petición, aplica middleware de autenticación si es necesario, ejecuta consultas en MySQL y devuelve un JSON.

Para publicar un anuncio, el usuario rellena el formulario y adjunta una imagen opcional. El frontend envía los datos al backend, que los valida y los almacena con estado pending. El moderador puede revisar el anuncio y cambiar el estado a approved o rejected.

Para consultar anuncios, el backend aplica filtros de categoría, comunidad y provincia, pagina los resultados y devuelve el listado al frontend.

La autenticación mediante JWT permite proteger rutas y diferenciar roles. El token se envía en la cabecera Authorization.

3.5.4. Seguridad lógica

La seguridad lógica se implementa en varios niveles.

Autenticación: el registro almacena la contraseña cifrada con bcrypt. El login verifica el hash y emite un token JWT.

Autorización: los tokens incluyen el id y el rol. Los middlewares de autenticación rechazan peticiones sin token o con token inválido. Los middlewares de roles permiten restringir acceso a administradores o moderadores.

Validación: Zod valida los esquemas de formularios en el frontend y express-validator valida los datos en el backend.

Protección HTTP: helmet añade cabeceras de seguridad. express-rate-limit limita las peticiones para evitar abuso. CORS se configura con los orígenes permitidos.

Prevención de inyección SQL: las consultas a MySQL usan parámetros con el pool de mysql2, evitando concatenación de cadenas.

Registro de actividad: la tabla admin_activity_logs almacena acciones relevantes junto con la IP para auditoría.

3.6. Análisis y diseño de la interfaz de usuario

3.6.1. Criterios de diseño

El diseño busca claridad, simplicidad y coherencia. Se ha utilizado Tailwind CSS para definir una paleta de colores, tipografías y espaciados consistentes.

La navegación es horizontal y se adapta a móvil mediante un menú colapsable. Los enlaces principales ofrecen acceso a anuncios, cultura, comunidad, sugerencias y propuestas.

Los formularios utilizan etiquetas claras, mensajes de error bajo los campos y botones descriptivos. Los estados de carga y error se muestran mediante componentes de retroalimentación.

Se evita el uso de emojis y elementos decorativos innecesarios. La paleta se centra en colores sobrios con acentos en los enlaces y botones de acción.

3.6.2. Pantallas representativas

A continuación se describen las pantallas principales.

Pantalla

Pantalla	Usuario	Objetivo	Elementos principales
Home	Todos	Consultar anuncios generales.	Listado, filtros, buscador.
Detalle de anuncio	Todos	Ver información completa.	Título, descripción, contacto, reporte.
Publicar anuncio	Usuario	Crear anuncio.	Formulario, adjunto, selects de territorio.
Comunidad	Todos	Ver publicaciones por provincia.	Publicaciones, comentarios, likes.
Sugerencias	Todos	Enviar sugerencias.	Formulario y listado.
Propuestas	Todos	Consultar propuestas.	Tarjetas y botón de apoyo.
Cultura y eventos	Todos	Ver actividades culturales.	Listado y filtros.
Recursos	Todos	Consultar recursos.	Listado por categoría.
Login	Visitante	Acceder al sistema.	Email, contraseña.
Registro	Visitante	Crear cuenta.	Formulario de registro.
Panel admin	Admin	Gestionar el sistema.	Dashboard, menús de módulos.

Panel moderador	Moderador	Revisar contenido.	Reportes, anuncios pendientes.
-----------------	-----------	--------------------	--------------------------------

Las capturas de pantalla no se han insertado en esta versión. Se recomienda añadirlas manualmente en una revisión posterior, con pie de figura correspondiente.

3.6.3. Diseño responsive

El diseño responsive se implementa mediante clases de Tailwind CSS. Se utiliza un enfoque mobile-first, de modo que las pantallas pequeñas reciben la disposición base y las superiores aprovechan el espacio adicional.

El menú de navegación se convierte en un botón de hamburguesa en móvil. Las grillas de anuncios pasan de una columna en móvil a tres o cuatro en escritorio. Los formularios ocupan el ancho completo en móvil y se limitan a un contenedor centrado en escritorio.

Las tablas del panel admin se adaptan con desplazamiento horizontal o mostrando columnas reducidas. Los modales y notificaciones se centran y escalan según el tamaño de pantalla.

Documento de Implementación, Pruebas e Implantación del sistema

4.1. Implementación

4.1.1. Estructura general del repositorio

El repositorio se organiza en carpetas principales que reflejan la arquitectura del proyecto.

La carpeta backend contiene el código del servidor, con src para el código fuente y dist para el compilado. La carpeta frontend contiene la aplicación Next.js con src. La carpeta database incluye init.sql con el esquema y datos iniciales. La carpeta docs/memoria alberga la documentación de la memoria.

En la raíz se encuentran docker-compose.yml, README.md, package.json y los archivos .env.example. El control de versiones se realiza con Git y el historial se almacena en GitHub.

Esta organización facilita la navegación del código y la revisión por parte del tribunal. Cada módulo se encuentra en una carpeta con responsabilidad clara.

4.1.2. Estructura del frontend

El frontend se estructura en frontend/src. La carpeta app contiene las páginas del sistema basadas en el enrutamiento de Next.js. Cada subcarpeta representa una ruta accesible desde el navegador.

La carpeta components organiza los componentes reutilizables. Se distinguen componentes de anuncios, comunidad, admin, ui, layout, forms e illustrations. Esta separación evita mezclar responsabilidades y facilita el mantenimiento.

Los hooks encapsulan la lógica de acceso a datos y la interacción con el backend. El contexto de autenticación gestiona el estado de sesión a nivel de aplicación.

Los archivos `lib` y `types` contienen utilidades, constantes y definiciones de TypeScript. El archivo de tipos permite compartir interfaces entre componentes.

Los estilos se configuran con Tailwind CSS. Los archivos de configuración de PostCSS y Tailwind se encuentran en la raíz de frontend.

4.1.3. Estructura del backend

El backend se organiza en `backend/src`. El archivo `index.ts` inicia el servidor, verifica la conexión a MySQL y crea los usuarios de demostración. El archivo `app.ts` configura middlewares, seguridad y rutas.

Los controladores, en `src/controllers`, implementan la lógica de cada recurso. Cada controlador suele corresponder a un dominio del sistema: `auth`, `anuncios`, `comunidad`, `sugerencias`, `propuestas`, `recursos`, `eventos`, `moderacion`, `admin`, etc.

Las rutas, en `src/routes`, definen los endpoints de la API REST. Cada router importa su controlador y expone los métodos HTTP necesarios.

Los middlewares, en `src/middleware`, gestionan autenticación, roles, errores y validaciones. La carpeta `config` incluye la conexión a base de datos y la centralización de variables de entorno.

Los `utils` incluyen funciones auxiliares, como el `logger`, la gestión de categorías, auditoría y territorios. Los `types` definen las interfaces de TypeScript extendidas.

4.1.4. Gestión de base de datos

La conexión a MySQL se configura en `backend/src/config/database.ts`. Se utiliza un pool de conexiones con `mysql2`, lo que permite reutilizar conexiones y mejorar el rendimiento.

El esquema se carga en el contenedor `mysql` mediante `database/init.sql`. El script se ejecuta automáticamente la primera vez que se levanta el volumen.

Los controladores ejecutan consultas parametrizadas. Esto significa que los valores se pasan separados de la sentencia SQL, lo que evita la inyección SQL.

La base de datos `citypaj` contiene datos de demostración: usuarios `admin`, `moderador` y `usuario`, anuncios de múltiples categorías, publicaciones de comunidad, sugerencias, propuestas, recursos, eventos y logs.

El endpoint `/health` y `/api/health` verifican que el backend ha conectado correctamente con MySQL. Devuelven el nombre de la base de datos, el estado y el tiempo de actividad.

4.1.5. Autenticación y roles

El sistema implementa autenticación con JWT. El registro almacena la contraseña cifrada con `bcrypt`. El login verifica el hash y emite un token de acceso y otro de refresco.

El token de acceso se envía en la cabecera `Authorization`. El middleware de autenticación decodifica el token y añade la información del usuario a la petición.

Los roles del sistema son `usuario`, `moderador` y `administrador`. Cada ruta protegida especifica el rol o roles permitidos. Si un usuario sin permisos intenta acceder, el backend devuelve un error 403.

El frontend guarda el token y utiliza el contexto de autenticación para saber si el usuario está autenticado. Los componentes protegidos redirigen al login si no hay sesión.

4.1.6. Gestión de anuncios

El módulo de anuncios es uno de los más completos. Permite consultar, crear, editar, guardar, reportar y moderar anuncios.

El controlador `anuncios-mysql.ts` implementa la consulta con filtros de categoría, comunidad, provincia, búsqueda y paginación. Utiliza una utilidad de categorías para detectar contenido cultural y separarlo del listado principal.

La publicación recoge título, descripción, categoría, subcategoría, territorio, modalidad, contacto y cartel. El cartel se sube al backend mediante el endpoint `/api/upload` con `multer`.

Los anuncios se crean con estado `pending` hasta que un moderador o administrador los apruebe. Este flujo evita la publicación automática de contenido inadecuado.

El detalle de anuncio muestra toda la información y permite al usuario guardar el anuncio como favorito, reportarlo o contactar.

4.1.7. Filtros por categoría y territorio

El sistema implementa filtros que permiten ajustar el contenido mostrado. La categoría distingue entre empleo, formación, vivienda, ocio, comunidad, cultura y otros tipos.

El filtro territorial utiliza los catálogos de comunidades y provincias. La provincia está vinculada a una comunidad autónoma, de modo que el selector puede mostrar solo las provincias correspondientes.

En el frontend, la barra de filtros y la página de búsqueda permiten combinar varios criterios. El backend recibe los parámetros por `query string` y construye la consulta SQL dinámicamente.

La utilidad `CATEGORIAS_CULTURA` identifica qué categorías pertenecen a la sección de cultura, de forma que los anuncios culturales se excluyen del home y se muestran en su pantalla específica.

4.1.8. Comunidad

La sección de comunidad permite crear publicaciones vinculadas a una provincia y a un tema. Los usuarios pueden comentar las publicaciones y dar me gusta.

La publicación puede ser anónima, en cuyo caso se solicita un nombre y se registra la dirección IP para moderación.

Los comentarios y publicaciones cuentan con estado de moderación. El panel de moderación permite ocultar o aprobar contenido.

La funcionalidad se encuentra implementada de forma parcial, puesto que permite la acción principal, a pesar de que algunos aspectos de la interfaz y la gestión de errores requieren mejoras.

4.1.9. Sugerencias y propuestas

El buzón de sugerencias permite enviar ideas, quejas o propuestas de mejora sin necesidad de estar registrado. Los campos son asunto, descripción, categoría y prioridad.

Las sugerencias se almacenan en la tabla `sugerencias` y se muestran en el panel de administración.

El módulo de propuestas muestra iniciativas ciudadanas y facilita a los usuarios apoyarlas. La tabla `propuestas_apoyos` almacena el apoyo vinculado a una propuesta.

La funcionalidad de propuestas se encuentra implementada de forma parcial, puesto que la visualización y el apoyo funcionan, pero la gestión avanzada requiere ampliación.

4.1.10. Cultura, eventos y recursos

La sección de cultura y eventos agrupa actividades culturales y eventos juveniles. Los anuncios con categorías culturales se filtran y se muestran en su propia pantalla.

La tabla `eventos` almacena título, descripción, fecha, provincia y otros datos. Los eventos se consultan a través del endpoint `/api/eventos`.

El módulo de recursos muestra recursos juveniles como convocatorias, guías o enlaces de interés. La tabla `recursos` almacena la información correspondiente.

La publicación de anuncios culturales integra el flujo general de anuncios, con la salvedad de que se excluyen del home para respetar la sección específica.

4.1.11. Administración y moderación

El panel de administración agrupa múltiples módulos: usuarios, anuncios, comunidad, cultura, estadísticas, tareas, agenda, mensajes, plantillas, comunicaciones e instituciones.

Cada módulo tiene una vista en el frontend y un conjunto de endpoints en el backend. Los datos se cargan mediante peticiones a la API.

El panel de moderación está destinado a revisar anuncios pendientes, reportes de contenido y publicaciones de comunidad. El moderador puede aprobar, rechazar, ocultar o marcar contenido.

Algunos paneles incluyen datos de demostración. La funcionalidad se encuentra implementada de forma parcial, puesto que permite la acción principal, a pesar de que algunos módulos requieren completar validaciones y estados.

4.1.12. Mensajería

El módulo de mensajería permite enviar mensajes internos entre usuarios del staff. La tabla `mensajes_staff` almacena asunto, cuerpo, remitente y destinatario.

La funcionalidad se encuentra implementada de forma parcial. La vista de mensajes y las plantillas existen, pero el envío real y las notificaciones requieren completarse.

4.1.13. Agenda institucional

La agenda institucional facilita a los administradores crear notas con fecha, título y cuerpo. La tabla `agenda_notas` almacena estas entradas.

La funcionalidad se encuentra implementada de forma parcial, puesto que la creación y consulta básicas funcionan, pero la integración con calendarios externos no está incluida.

4.1.14. Comunicaciones institucionales

El módulo de comunicaciones institucionales permite preparar borradores de mensajes a entidades. Utiliza plantillas predefinidas para asunto y cuerpo.

La funcionalidad se encuentra implementada de forma parcial. Los borradores y las plantillas están presentes, pero el envío real depende de una configuración de correo no activa en el entorno local.

4.1.15. Dockerización del proyecto

La dockerización se define en `docker-compose.yml`. El archivo declara tres servicios: `mysql`, `backend` y `frontend`.

El servicio `mysql` utiliza la imagen `mysql:8.0`, crea la base de datos `citypaj`, el usuario `citypaj_user` y carga `database/init.sql`. El `healthcheck` comprueba que la tabla `usuarios` existe.

El servicio `backend` se construye desde `backend/Dockerfile` con `target production`. Expone el puerto 3002 y espera a que MySQL esté saludable.

El servicio `frontend` se construye desde `frontend/Dockerfile`. Expone el puerto 3001 y depende del `backend`.

La red interna de Docker permite que los contenedores se comuniquen por nombre. El `frontend` accede al `backend` en `http://backend:3002` y el navegador accede en `http://localhost:3002`.

4.2. Instalación, despliegue y configuración

4.2.1. Requisitos previos

Los requisitos previos son mínimos. Solo es necesario disponer de Docker Desktop en ejecución y de Git para clonar el repositorio.

No es necesario instalar Node.js ni MySQL de forma local, puesto que ambos corren dentro de contenedores. La primera ejecución descarga las imágenes de Docker Hub y construye los contenedores.

El ordenador debe disponer de suficiente espacio en disco para las imágenes de Docker y de memoria RAM para ejecutar tres contenedores simultáneamente.

4.2.2. Variables de entorno

Los archivos `.env.example` y `backend/.env.example` contienen los valores por defecto. Para la primera ejecución no es necesario modificar nada.

Las variables principales son: `NODE_ENV`, `PORT`, `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `JWT_SECRET`, `JWT_REFRESH_SECRET`, `CORS_ORIGINS` y las credenciales de los usuarios `demo`.

Estos valores son seguros únicamente para demostración local. En un entorno de producción deberían cambiarse por secretos reales y no exponerse en el repositorio.

4.2.3. Ejecución con Docker Compose

Para levantar el proyecto se ejecutan los siguientes comandos desde la raíz del repositorio:

```
git clone https://github.com/carmendmv/citypaj
cd
docker compose up --build
```

La primera ejecución descarga imágenes, instala dependencias, compila el backend y el frontend y carga la base de datos. Puede tardar varios minutos.

Cuando todos los contenedores estén saludables, el frontend estará disponible en <http://localhost:3001>, el backend en <http://localhost:3002> y el health check en <http://localhost:3002/health>.

4.2.4. Ejecución manual

a pesar de que el proyecto está pensado para Docker, asimismo es posible ejecutar los servicios manualmente. Para ello se necesita una instancia MySQL local con la base de datos citypaj cargada.

Se ejecuta `npm install` en backend y frontend, se configuran las variables de entorno y se lanzan `npm run dev:backend` y `npm run dev:frontend` en terminales separadas.

Esta forma de ejecución es útil para desarrollo, pero requiere más configuración. El repositorio no documenta este modo en detalle.

4.2.5. Acceso a la aplicación

Una vez levantada, la aplicación es accesible desde el navegador en las siguientes URLs:

Servicio

Servicio	URL
Frontend	http://localhost:3001
Backend	http://localhost:3002
Health check	http://localhost:3002/health
Base de datos	<code>mysql://citypaj_user:citypaj_password@localhost:3306/citypaj</code>

Los usuarios de demostración permiten probar los roles: `admin@citypaj.local` con `Admin1234`, `moderador@citypaj.local` con `Moderador1234` y `usuario@citypaj.local` con `Usuario1234`.

4.2.6. Resolución de problemas frecuentes

Si Docker Desktop no responde, se debe comprobar que el motor Docker está en ejecución antes de lanzar `docker compose up`.

Si el backend indica que no se pudo conectar a la base de datos, es posible que MySQL no esté listo. La configuración de `depends_on` con `condition: service_healthy` hace que el backend espere y se reinicie si es necesario.

Si el frontend no encuentra el backend, se debe verificar que los contenedores están en ejecución y que el backend responde en <http://localhost:3002>.

Si los puertos 3001 o 3002 están ocupados, se deben detener los procesos que los usan o modificar los puertos en `docker-compose.yml`.

4.3. Pruebas

4.3.1. Plan de pruebas

El plan de pruebas cubre las funcionalidades principales del sistema. Las pruebas se han realizado de forma manual y con los usuarios demo.

Cada prueba incluye los pasos, el resultado esperado, el resultado obtenido y el estado. Si una prueba no se ha ejecutado en el entorno actual, se marca como No ejecutada.

4.3.2. Pruebas funcionales

ID

ID	Prueba	Pasos	Esperado	Obtenido	Estado
PF-01	Registro	Rellenar formulario y enviar.	Cuenta creada.	No ejecutada	No ejecutada
PF-02	Login	Introducir credenciales demo.	Token y acceso.	No ejecutada	No ejecutada
PF-03	Consultar anuncios	Cargar home.	Listado visible.	No ejecutada	No ejecutada
PF-04	Filtros por categoría	Seleccionar categoría.	Listado filtrado.	No ejecutada	No ejecutada
PF-05	Filtros por territorio	Seleccionar provincia.	Listado filtrado.	No ejecutada	No ejecutada
PF-06	Publicar anuncio	Rellenar formulario.	Anuncio creado pending.	No ejecutada	No ejecutada
PF-07	Guardar favorito	Pulsar favorito.	Aparece en guardados.	No ejecutada	No ejecutada
PF-08	Reportar anuncio	Enviar reporte.	Reporte registrado.	No ejecutada	No ejecutada
PF-09	Comunidad	Crear publicación.	Publicación visible.	No ejecutada	No ejecutada
PF-10	Sugerencias	Enviar sugerencia.	Sugerencia registrada.	No ejecutada	No ejecutada
PF-11	Propuestas	Consultar propuestas.	Listado visible.	No ejecutada	No ejecutada
PF-12	Cultura y eventos	Acceder a cultura.	Eventos visibles.	No ejecutada	No ejecutada
PF-13	Recursos	Acceder a recursos.	Listado visible.	No ejecutada	No ejecutada
PF-14	Panel admin	Login admin.	Acceso a módulos.	No ejecutada	No ejecutada

PF-15	Panel moderador	Login moderador.	Acceso a reportes.	No ejecutada	No ejecutada
-------	-----------------	------------------	--------------------	--------------	--------------

4.3.3. Pruebas de validación

Las pruebas de validación verifican que los formularios rechazan datos incorrectos. Se comprueban campos vacíos, formatos de email, longitudes máximas y valores obligatorios.

El sistema utiliza Zod en el frontend y express-validator en el backend. Estas herramientas validan los datos en dos capas.

Las pruebas no se han ejecutado en el entorno actual. Se recomienda completarlas en una revisión posterior.

4.3.4. Pruebas de base de datos

Las pruebas de base de datos verifican que el esquema se carga correctamente, que las conexiones son estables y que las consultas devuelven los datos esperados.

El endpoint /health realiza una consulta SELECT 1 a MySQL y devuelve el estado. Si la base de datos no responde, el endpoint devuelve 503.

Las pruebas de consistencia de datos, como la existencia de claves foráneas, requieren ejecutar la aplicación y consultar directamente las tablas.

4.3.5. Pruebas de seguridad básica

Las pruebas de seguridad básica incluyen el intento de acceso a rutas protegidas sin token, el envío de datos inválidos y la comprobación de que las contraseñas no se almacenan en texto plano.

Se ha verificado que el backend utiliza bcrypt para las contraseñas y que las rutas admin requieren autenticación.

Las pruebas de inyección SQL se apoyan en el uso de consultas parametrizadas con mysql2.

4.3.6. Pruebas de diseño responsive

Las pruebas de diseño responsive se realizan reduciendo el tamaño de la ventana del navegador. Se verifica que el menú se colapsa, las grillas se ajustan y los formularios no desbordan.

No se han ejecutado pruebas automáticas de responsive. La verificación se realiza de forma manual.

4.3.7. Pruebas de Docker

Las pruebas de Docker verifican que docker compose up --build levanta los tres servicios sin errores y que el frontend y backend responden.

El reinicio desde cero se prueba con docker compose down -v seguido de docker compose up --build, comprobando que la base de datos se recrea.

No se han ejecutado estas pruebas en el entorno de generación de la memoria.

4.3.8. Resultado de las pruebas

El resultado de las pruebas es parcial. El código y la configuración están preparados para ser probados, pero las pruebas manuales no se han ejecutado en el entorno de redacción.

El plan de pruebas queda documentado con el fin de que el tribunal o el usuario pueda completarlo en una ejecución local.

4.4. Manual de usuario

4.4.1. Descripción general

CityPAJ es una aplicación web para consultar y publicar información juvenil organizada por territorio. El manual explica los pasos básicos de uso.

4.4.2. Puesta en marcha

Para poner en marcha la aplicación se clona el repositorio y se ejecuta `docker compose up --build`. Después se espera a que los contenedores estén saludables y se accede a `http://localhost:3001`.

4.4.3. Registro e inicio de sesión

Los usuarios pueden registrarse desde el enlace correspondiente. Para probar los roles se utilizan las credenciales de demostración: `admin@citypaj.local`, `moderador@citypaj.local` y `usuario@citypaj.local`.

4.4.4. Navegación por anuncios

El home muestra el listado de anuncios. El usuario puede desplazarse, usar el buscador y pulsar un anuncio para ver el detalle.

4.4.5. Publicación de anuncios

Desde la página de publicar, el usuario rellena el formulario, selecciona la categoría y el territorio, y opcionalmente adjunta un cartel. Tras enviar, el anuncio queda en estado pendiente.

4.4.6. Uso de filtros

La barra de filtros permite seleccionar categoría, comunidad autónoma y provincia. Los resultados se actualizan sin recargar la página.

4.4.7. Comunidad

La comunidad muestra publicaciones por provincia. El usuario puede crear una publicación, comentar y dar me gusta. asimismo es posible participar de forma anónima.

4.4.8. Sugerencias

El buzón de sugerencias permite enviar ideas o avisos. No es necesario estar registrado.

4.4.9. Propuestas

La página de propuestas muestra iniciativas ciudadanas. El usuario puede consultar el detalle y apoyar la propuesta.

4.4.10. Cultura y eventos

La sección de cultura muestra eventos y anuncios culturales. Se accede desde el menú principal.

4.4.11. Recursos

La página de recursos lista convocatorias, guías y enlaces de interés para jóvenes.

4.4.12. Panel de administración

El panel de administración requiere el rol admin. Permite gestionar usuarios, anuncios, tareas, agenda, mensajes, plantillas, comunicaciones e instituciones.

4.4.13. Panel de moderación

El panel de moderación requiere el rol moderador. Permite revisar anuncios pendientes, reportes y contenido de comunidad.

4.4.14. Mensajes de error frecuentes

Si aparece un error de conexión, se recomienda comprobar que Docker Desktop está en ejecución y que el backend responde en `http://localhost:3002`.

Si el login falla, se debe verificar que se ha introducido el correo y la contraseña correctamente y que el backend ha conectado con MySQL.

Si la publicación de un anuncio no se guarda, se recomienda revisar los campos obligatorios y la categoría seleccionada.

4.4.15. Copias de seguridad

La base de datos es posible respaldar exportando el volumen de Docker o copiando el archivo `database/init.sql`. Para un entorno real se recomendaría un sistema de backups automáticos.

4.5. Ayuda integrada

El proyecto no incluye una sección de ayuda contextual integrada en la interfaz. La ayuda se proporciona a través del manual de usuario de la memoria y del archivo `README.md` del repositorio.

El `README.md` describe los comandos de instalación, las URLs de acceso, las credenciales de demostración y los pasos básicos de uso. Esta documentación sirve como referencia para el tribunal y para futuros usuarios.

Documento de Cierre

5.1. Resultados obtenidos

El proyecto ha producido una aplicación web fullstack dockerizada con frontend, backend y base de datos. El sistema permite el registro de usuarios, la publicación y consulta de anuncios, la gestión de favoritos, el envío de sugerencias, la consulta de propuestas, la comunidad por provincias, la sección de cultura y eventos, y paneles de administración y moderación.

La dockerización local permite ejecutar la aplicación con un solo comando y facilita la revisión. El README describe el despliegue y el repositorio contiene el esquema de base de datos.

Algunas funcionalidades se encuentran implementadas de forma parcial, especialmente en el área de mensajería, comunicaciones institucionales y algunos módulos del panel admin con datos de demostración.

5.2. Conclusiones

El proyecto ha permitido poner en práctica las tecnologías aprendidas durante el ciclo. La combinación de Next.js, Express y MySQL ha supuesto un desafío técnico que ha ayudado a comprender el flujo de una aplicación web real.

La dockerización ha sido una decisión tardía pero útil para la entrega. El filtrado territorial y la moderación de contenidos han sido los módulos más complejos.

desde la perspectiva del estudiante, se ha adquirido la importancia de planificar la arquitectura desde el inicio, de documentar las decisiones técnicas y de probar el despliegue antes de la entrega.

El valor del proyecto radica en ofrecer una herramienta de centralización de información juvenil, con un enfoque territorial y participativo.

5.3. Diario de bitácora

Periodo

Periodo	Trabajo realizado	Evidencia	Observaciones
Enero-Febrero	Análisis inicial, prototipado y configuración del repositorio.	Estructura básica y README inicial.	Aprendizaje de Next.js App Router.
Marzo-Abril	Conexión con MySQL, desarrollo de autenticación y anuncios.	Migraciones y controladores.	Dificultades con tipos.
Mayo	Paneles admin, moderación, comunidad, sugerencias y propuestas.	Rutas de admin y moderador.	numerosos módulos en paralelo.
Junio-Agosto	Dockerización, limpieza, README y memoria.	Docker compose y docs.	Revisión final y ajustes.

5.4. Temporalización real

El desarrollo se ha extendido desde enero hasta agosto. La intensidad fue mayor durante los últimos meses, cuando se completó la integración de Docker y la limpieza del repositorio.

La fase de análisis y diseño se completó en las primeras semanas. La implementación se aceleró a partir de marzo. El cierre incluyó la memoria y la preparación de la entrega.

5.5. Desviación respecto a la planificación inicial

Tarea

Tarea	Planificado	Real	Desviación	Explicación
Docker	8	12	+4	Configuración del proxy entre contenedores.
Moderación	10	15	+5	Se añadieron filtros y acciones masivas.
Pruebas	10	6	-4	Se priorizó la funcionalidad.
Documentación	12	14	+2	Ampliación del README y memoria.

5.6. Conocimientos adquiridos

Durante el proyecto se han adquirido conocimientos en el desarrollo de aplicaciones web fullstack con Next.js, React, TypeScript, Express, MySQL y Docker.

se ha adquirido a diseñar una API REST, a gestionar autenticación con JWT, a validar formularios con Zod y a dockerizar una aplicación completa.

asimismo se ha comprendido la importancia de la organización del código, la documentación y las pruebas para proyectos colaborativos y académicos.

5.7. Limitaciones del proyecto

El proyecto prioriza el entorno local y la demostración. Algunas funciones del panel admin se presentan con datos de demostración. La mensajería real, el envío de correos y las comunicaciones institucionales no están completamente operativas.

La aplicación no incluye un sistema de recuperación de contraseña completamente funcional, a pesar de que existe la página de recuperar contraseña.

Las pruebas automatizadas son limitadas. El proyecto se ha probado principalmente de forma manual.

La internacionalización fue eliminada del repositorio, por lo que la aplicación solo está disponible en castellano.

5.8. Posibles mejoras futuras

Las posibles mejoras futuras incluyen completar la mensajería interna con notificaciones, implementar pruebas automáticas, mejorar la accesibilidad, optimizar consultas con índices, añadir estadísticas en tiempo real, preparar un despliegue en servidor propio y mejorar la gestión de errores.

asimismo sería útil completar el envío real de correos, integrar un sistema de notificaciones y ampliar la comunidad con más opciones de participación.

Bibliografía

Anexos

7.1. Enlace al repositorio

El repositorio del proyecto está disponible en: <https://github.com/carmendmv/citypaj>

7.2. Estructura del repositorio

El repositorio contiene las siguientes carpetas principales: backend, frontend, database, docs/memoria y los archivos de configuración docker-compose.yml, README.md, .env.example y package.json.

Carpeta/Archivo

Carpeta/Archivo	Descripción
backend/	Código fuente del servidor Express y TypeScript.
frontend/	Aplicación Next.js, React y TypeScript.
database/	Script init.sql con esquema y datos iniciales.
docs/memoria/	Documentación de la memoria.
docker-compose.yml	Orquestación de contenedores.
README.md	Guía de instalación y uso.
.env.example	Variables de entorno de ejemplo.
package.json	Scripts y dependencias raíz.

7.3. Capturas de pantalla

A continuación se incluyen capturas de pantalla representativas del frontend, generadas a partir del despliegue local de la aplicación. Estas imágenes ilustran el aspecto visual del proyecto en resoluciones de escritorio y dispositivo móvil.

Captura 1. Vista de escritorio de CityPAJ



Captura 2. Vista móvil de CityPAJ



7.4. Diagramas SVG generados

En la carpeta docs/memoria/diagramas se han generado los siguientes diagramas en formato SVG:

Archivo

Archivo	Descripción
01_arquitectura_general.svg	Arquitectura general de tres capas.
02_flujo_usuario_frontend_backend_bbdd.svg	Flujo de comunicación entre capas.

03_modelo_entidad_relacion.svg	Entidades principales del sistema.
04_modelo_relacional.svg	Modelo relacional de tablas.
05_casos_uso.svg	Actores y casos de uso.
06_secuencia_login.svg	Secuencia de autenticación.
07_secuencia_publicacion_anuncio.svg	Secuencia de publicación de anuncio.
08_estados_anuncio.svg	Estados de un anuncio.
09_arquitectura_docker.svg	Arquitectura de contenedores.
10_modulos_frontend.svg	Módulos del frontend.
11_modulos_backend.svg	Módulos del backend.
12_flujo_moderacion.svg	Flujo de moderación.
13_flujo_sugerencias_propuestas.svg	Flujo de sugerencias y propuestas.
14_flujo_comunidad.svg	Flujo de comunidad.

Ilustraciones generadas

Figura 1. 01 arquitectura general

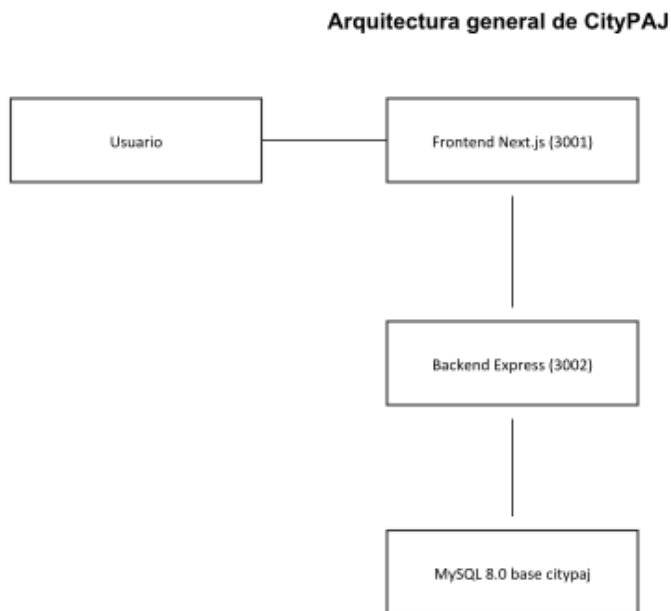


Figura 2. 02 flujo usuario frontend backend bbdd

Flujo usuario-frontend-backend-base de datos

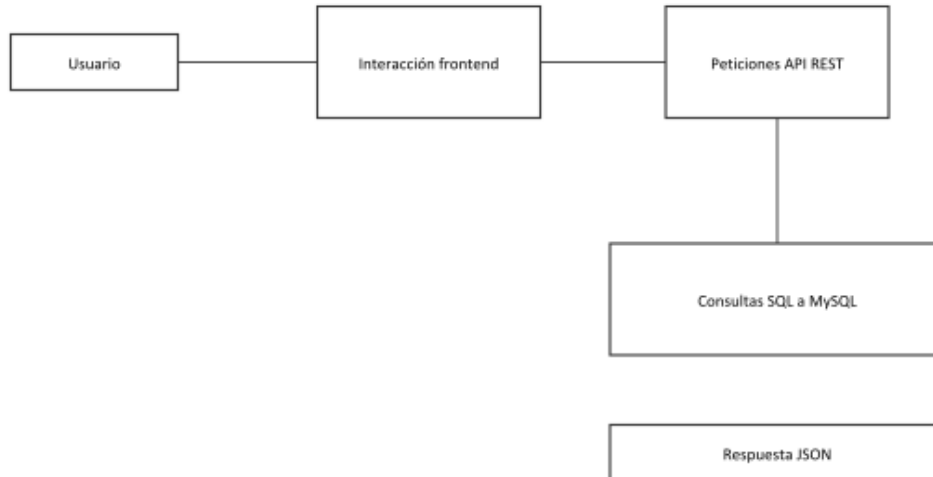


Figura 3. 03 modelo entidad relacion

Modelo entidad-relación

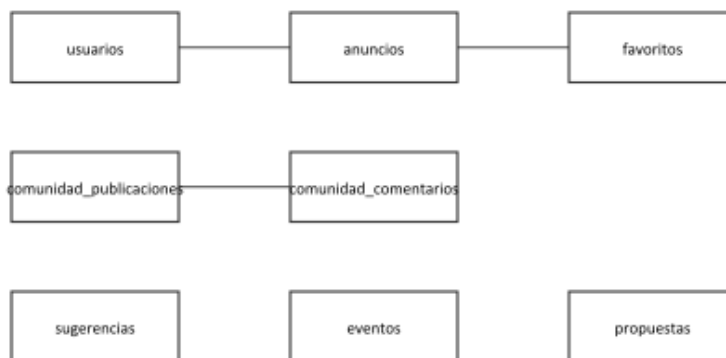


Figura 4. 04 modelo relacional

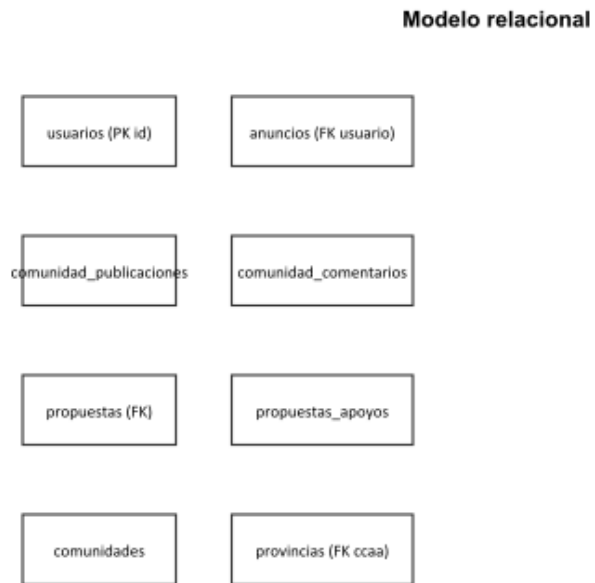


Figura 5. 05 casos uso

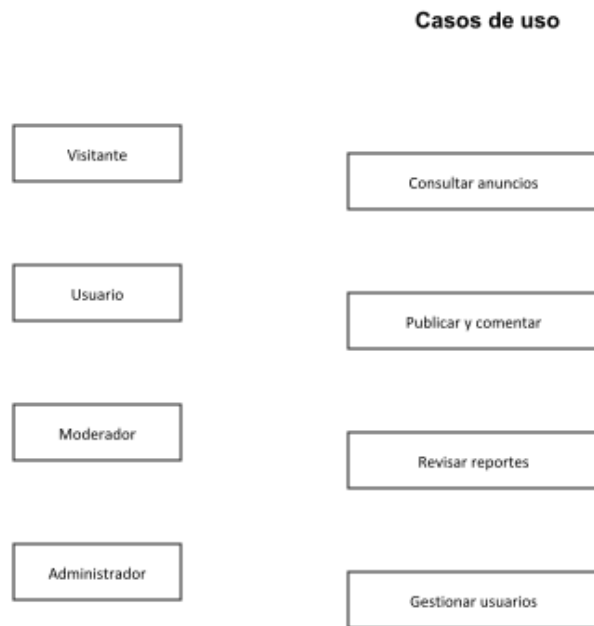


Figura 6. 06 secuencia login

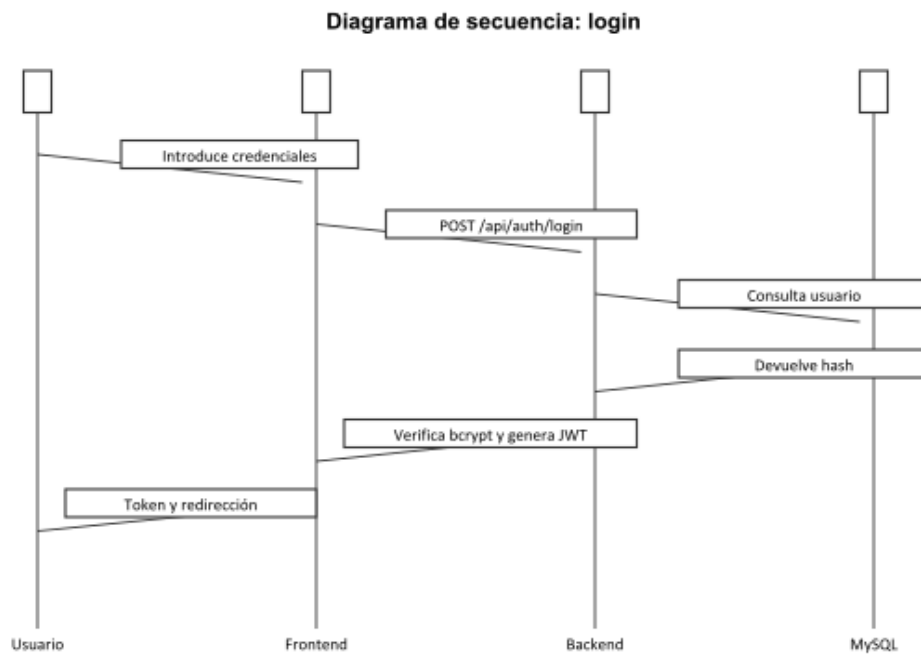


Figura 7. 07 secuencia publicacion anuncio

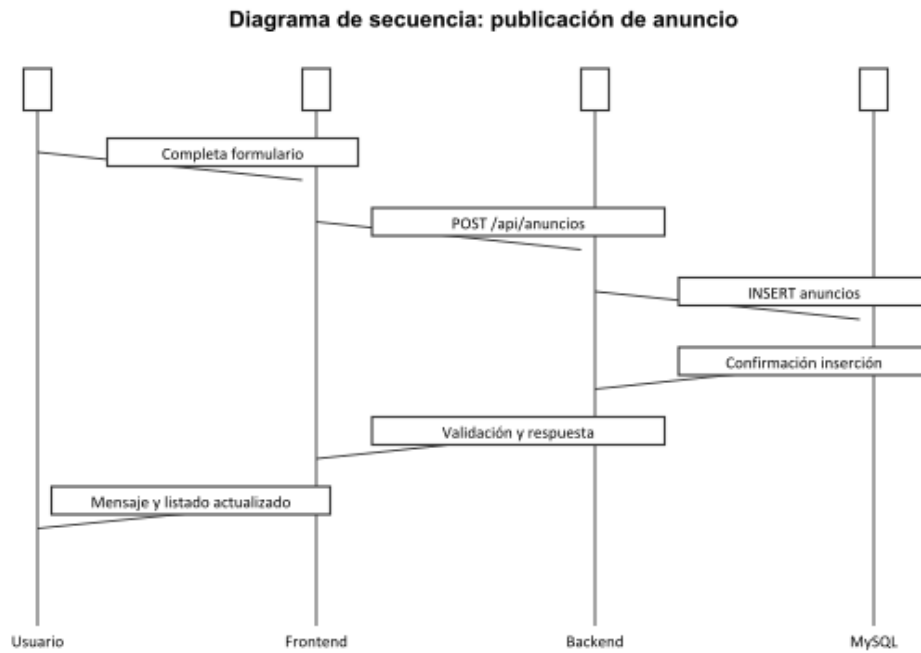


Figura 8. 08 estados anuncio

Diagrama de estados de un anuncio



Figura 9. 09 arquitectura docker

Arquitectura Docker

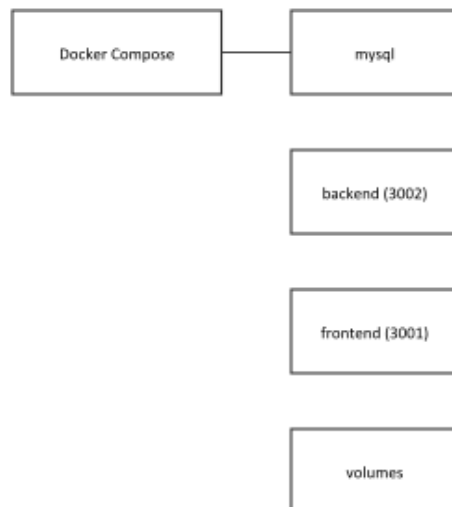


Figura 10. 10 modulos frontend

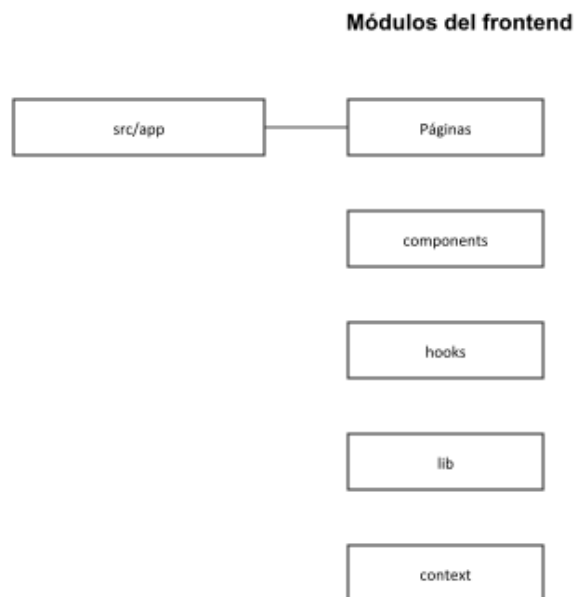


Figura 11. 11 modulos backend

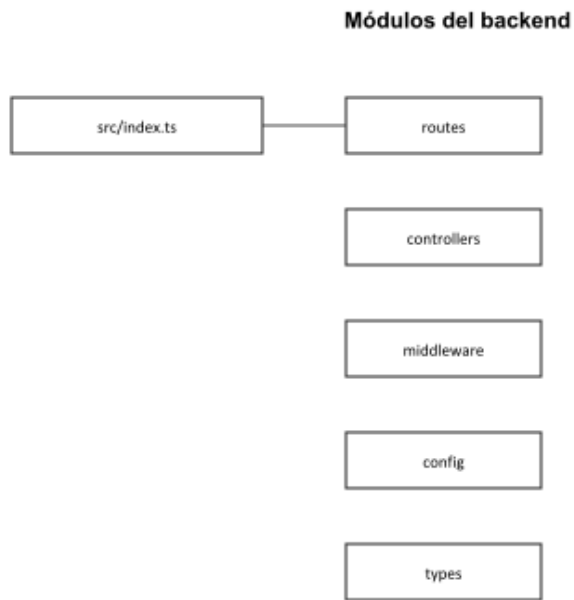


Figura 12. 12 flujo moderacion

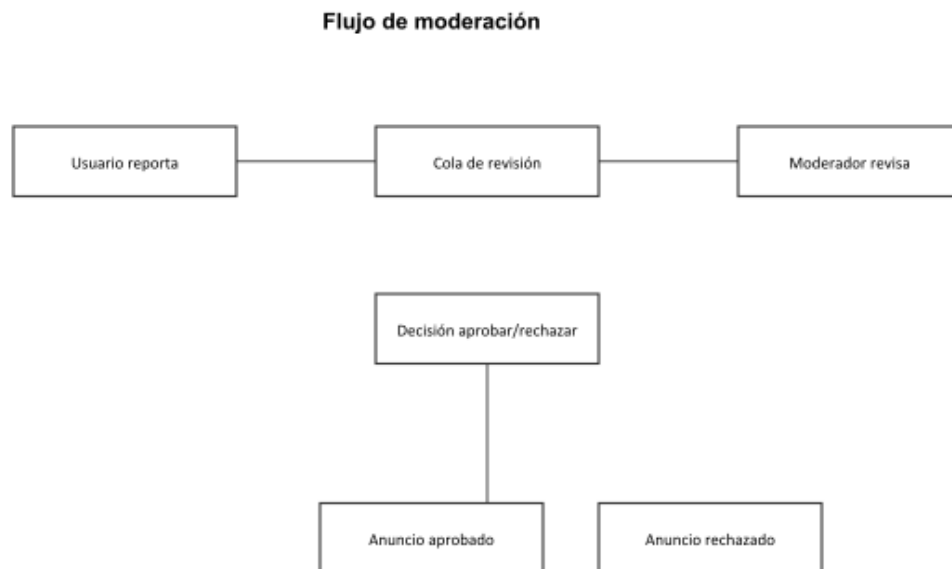


Figura 13. 13 flujo sugerencias propuestas

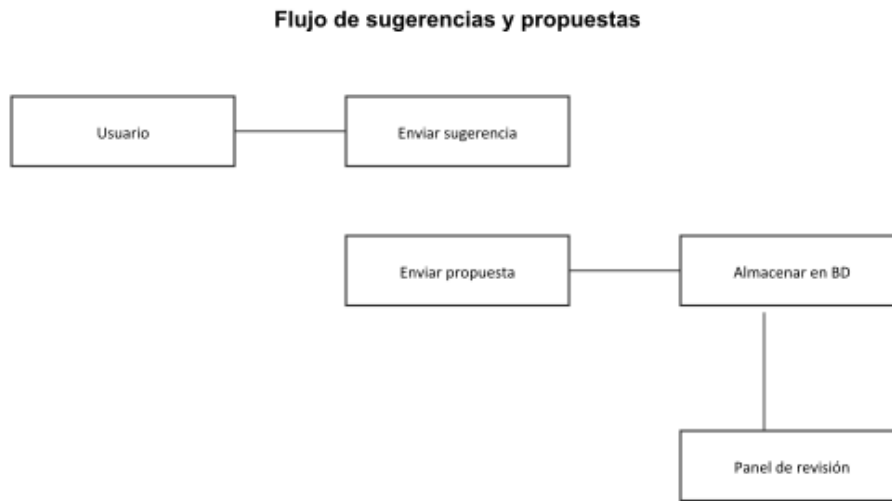


Figura 14. 14 flujo comunidad



7.5. Script de base de datos

El script de base de datos es `database/init.sql`. Crea el esquema completo de la base de datos citypaj e inserta datos iniciales. No se incluye el código completo en la memoria para no alargar el documento; el repositorio lo contiene.

7.6. Docker Compose

El archivo `docker-compose.yml` define los servicios `mysql`, `backend` y `frontend`. La configuración completa se encuentra en el repositorio.

A continuación se muestra un resumen de los servicios:

Servicio

Servicio	Imagen/Build	Puerto	Dependencia
mysql	mysql:8.0	3306	-
backend	Build backend/Dockerfile	3002	mysql
frontend	Build frontend/Dockerfile	3001	backend

7.7. Variables de entorno de ejemplo

Los archivos `.env.example` y `backend/.env.example` contienen los valores por defecto. Se muestra un resumen:

Variable

Variable	Valor ejemplo	Descripción
NODE_ENV	development	Entorno de ejecución.
PORT	3002	Puerto del backend.
DB_HOST	mysql	Host de MySQL en Docker.
DB_NAME	citypaj	Base de datos.
DB_USER	citypaj_user	Usuario de MySQL.
DB_PASSWORD	citypaj_password	Contraseña de MySQL.
JWT_SECRET	citypaj_secret_demo	Secreto JWT.
DEMO_ADMIN_EMAIL	admin@citypaj.local	Usuario admin demo.

7.8. Endpoints principales

La API REST expone los siguientes grupos de endpoints. Cada grupo corresponde a un router en backend/src/routes.

Ruta base

Ruta base	Recurso
/api/auth	Autenticación
/api/anuncios	Anuncios
/api/upload	Subida de archivos
/api/admin	Administración
/api/usuarios	Usuarios
/api/moderacion	Moderación
/api/reportes	Reportes
/api/sugerencias	Sugerencias
/api/comunidad	Comunidad
/api/propuestas	Propuestas
/api/recursos	Recursos
/api/eventos	Eventos
/api/estadisticas	Estadísticas
/api/territorios	Comunidades autónomas
/api/provincias	Provincias
/health	Health check del backend

7.9. Commits relevantes

El historial de Git contiene los commits que reflejan la evolución del proyecto. A continuación se listan algunos de los más representativos:

Hash

Hash	Fecha	Mensaje
329ef1a	2026-08-10	readme definitivo
39b908b	2026-08-10	Elimina scripts de prueba y archivos temporales
a66cdfb	2026-08-10	Permite publicar anuncios con todas las categorías y adjuntar carteles
5a00d2e	2026-08-10	Excluye anuncios culturales del home y anade utilidad de categorías
2e21e98	2026-08-10	Configura proxy interno del frontend hacia el backend en Docker
ebdab2b	2026-08-10	Implementa API de anuncios, filtro de cultura y utilidad de categorías
4362e19	2026-08-10	Configura Express, seguridad, conexion a MySQL y subida de imagenes
b38ea41	2026-08-10	Anade esquema y datos iniciales de MySQL
c737001	2026-08-10	Configura Docker, variables de entorno y scripts de build

7.10. Manual de instalación resumido

Para instalar y ejecutar el proyecto:

```
git clone https://github.com/carmendmv/citypaj
cd citypaj
docker compose up --build
```

Acceso: <http://localhost:3001>. Usuarios demo: admin@citypaj.local / Admin1234, moderador@citypaj.local / Moderador1234, usuario@citypaj.local / Usuario1234.

7.11. Estructura real del repositorio

Esta sección recoge las rutas reales del frontend y del backend. Los datos se han extraído del repositorio en la rama main.

Ruta relativa

Ruta relativa	Área
frontend\src\app\acceder\page.tsx	Ruta del frontend
frontend\src\app\admin\acceder\page.tsx	Ruta del frontend
frontend\src\app\admin\actividad\page.tsx	Ruta del frontend
frontend\src\app\admin\agenda\page.tsx	Ruta del frontend
frontend\src\app\admin\anuncios\page.tsx	Ruta del frontend
frontend\src\app\admin\calendario\page.tsx	Ruta del frontend
frontend\src\app\admin\comunicaciones\[id]\page.tsx	Ruta del frontend

frontend\src\app\admin\comunicaciones\nueva\page.tsx	Ruta del frontend
frontend\src\app\admin\comunicaciones\page.tsx	Ruta del frontend
frontend\src\app\admin\comunidad\page.tsx	Ruta del frontend
frontend\src\app\admin\cultura\page.tsx	Ruta del frontend
frontend\src\app\admin\estadisticas\page.tsx	Ruta del frontend
frontend\src\app\admin\instituciones\contactos\page.tsx	Ruta del frontend
frontend\src\app\admin\layout.tsx	Ruta del frontend
frontend\src\app\admin\mensajes\[id]\page.tsx	Ruta del frontend
frontend\src\app\admin\mensajes\page.tsx	Ruta del frontend
frontend\src\app\admin\moderadores\page.tsx	Ruta del frontend
frontend\src\app\admin\necesidades\page.tsx	Ruta del frontend
frontend\src\app\admin\page.tsx	Ruta del frontend
frontend\src\app\admin\plantillas\[id]\page.tsx	Ruta del frontend
frontend\src\app\admin\plantillas\page.tsx	Ruta del frontend
frontend\src\app\admin\sugerencias\page.tsx	Ruta del frontend
frontend\src\app\admin\tareas\page.tsx	Ruta del frontend
frontend\src\app\admin\usuarios\page.tsx	Ruta del frontend
frontend\src\app\anuncios\[id]\editar\page.tsx	Ruta del frontend
frontend\src\app\anuncios\[id]\page.tsx	Ruta del frontend
frontend\src\app\anuncios\comunidad\[comunidad]\page.tsx	Ruta del frontend
frontend\src\app\anuncios\page.tsx	Ruta del frontend
frontend\src\app\api\anuncios\[id]\moderar\route.ts	Ruta del frontend
frontend\src\app\api\anuncios\[id]\reportes\route.ts	Ruta del frontend
frontend\src\app\api\anuncios\[id]\route.ts	Ruta del frontend
frontend\src\app\api\anuncios\guardar\route.ts	Ruta del frontend
frontend\src\app\api\anuncios\moderacion\route.ts	Ruta del frontend
frontend\src\app\api\anuncios\publico\route.ts	Ruta del frontend
frontend\src\app\api\anuncios\reportar\route.ts	Ruta del frontend
frontend\src\app\api\anuncios\route.ts	Ruta del frontend
frontend\src\app\api\anuncios\search\route.ts	Ruta del frontend
frontend\src\app\api\auth\login\route.ts	Ruta del frontend
frontend\src\app\api\auth\logout\route.ts	Ruta del frontend
frontend\src\app\api\auth\refresh\route.ts	Ruta del frontend
frontend\src\app\api\auth\register\route.ts	Ruta del frontend

frontend\src\app\api\estadisticas\home\route.ts	Ruta del frontend
frontend\src\app\api\eventos\route.ts	Ruta del frontend
frontend\src\app\api\propuestas\route.ts	Ruta del frontend
frontend\src\app\api\recursos\route.ts	Ruta del frontend
frontend\src\app\api\sugerencias\estadisticas\route.ts	Ruta del frontend
frontend\src\app\api\sugerencias\route.ts	Ruta del frontend
frontend\src\app\ayudas\page.tsx	Ruta del frontend
frontend\src\app\buscar\page.tsx	Ruta del frontend
frontend\src\app\buzon-sugerencias\page.tsx	Ruta del frontend
frontend\src\app\comunidad\[provincia]\page.tsx	Ruta del frontend
frontend\src\app\comunidad\crear\page.tsx	Ruta del frontend
frontend\src\app\comunidad\page.tsx	Ruta del frontend
frontend\src\app\comunidad\publicacion\[id]\page.tsx	Ruta del frontend
frontend\src\app\contacto\page.tsx	Ruta del frontend
frontend\src\app\cultura\page.tsx	Ruta del frontend
frontend\src\app\empleo\page.tsx	Ruta del frontend
frontend\src\app\eventos\page.tsx	Ruta del frontend
frontend\src\app\formacion\page.tsx	Ruta del frontend
frontend\src\app\globals.css	Ruta del frontend
frontend\src\app\guardados\page.tsx	Ruta del frontend
frontend\src\app\instituciones\page.tsx	Ruta del frontend
frontend\src\app\layout.tsx	Ruta del frontend
frontend\src\app\mi-perfil\page.tsx	Ruta del frontend
frontend\src\app\mis-anuncios\page.tsx	Ruta del frontend
frontend\src\app\moderador\login\page.tsx	Ruta del frontend
frontend\src\app\moderador\page.tsx	Ruta del frontend
frontend\src\app\ocio\page.tsx	Ruta del frontend
frontend\src\app\page.tsx	Ruta del frontend
frontend\src\app\perfil\page.tsx	Ruta del frontend
frontend\src\app\privacidad\page.tsx	Ruta del frontend
frontend\src\app\propuestas\page.tsx	Ruta del frontend
frontend\src\app\providers.tsx	Ruta del frontend
frontend\src\app\publicar\cultura\page.tsx	Ruta del frontend
frontend\src\app\publicar\page.tsx	Ruta del frontend
frontend\src\app\recuperar-contrasena\page.tsx	Ruta del frontend

frontend\src\app\recursos\page.tsx	Ruta del frontend
frontend\src\app\servicios\page.tsx	Ruta del frontend
frontend\src\app\terminos\page.tsx	Ruta del frontend
frontend\src\app\vivienda\page.tsx	Ruta del frontend

Ruta relativa

Ruta relativa	Área
backend\src\app.ts	Ruta del backend
backend\src\config\database.ts	Ruta del backend
backend\src\config\index.ts	Ruta del backend
backend\src\controllers\admin-agenda.ts	Ruta del backend
backend\src\controllers\admin-anuncios.ts	Ruta del backend
backend\src\controllers\admin-comunicaciones.ts	Ruta del backend
backend\src\controllers\admin-comunidad.ts	Ruta del backend
backend\src\controllers\admin-contactos.ts	Ruta del backend
backend\src\controllers\admin-destinatarios.ts	Ruta del backend
backend\src\controllers\admin-estadisticas.ts	Ruta del backend
backend\src\controllers\admin-logs.ts	Ruta del backend
backend\src\controllers\admin-mensajes.ts	Ruta del backend
backend\src\controllers\admin-plantillas.ts	Ruta del backend
backend\src\controllers\admin-resumen.ts	Ruta del backend
backend\src\controllers\admin-tareas.ts	Ruta del backend
backend\src\controllers\admin-usuarios.ts	Ruta del backend
backend\src\controllers\anuncios-mysql.ts	Ruta del backend
backend\src\controllers\auth-simple.ts	Ruta del backend
backend\src\controllers\auth.ts	Ruta del backend
backend\src\controllers\comunidad.ts	Ruta del backend
backend\src\controllers\estadisticas.ts	Ruta del backend
backend\src\controllers\eventos.ts	Ruta del backend
backend\src\controllers\moderacion.ts	Ruta del backend
backend\src\controllers\propuestas.ts	Ruta del backend
backend\src\controllers\recursos.ts	Ruta del backend
backend\src\controllers\reportes.ts	Ruta del backend
backend\src\controllers\sugerencias.ts	Ruta del backend
backend\src\controllers\upload.ts	Ruta del backend
backend\src\controllers\usuarios.ts	Ruta del backend

backend\src\data\anuncios.json	Ruta del backend
backend\src\data\usuarios.json	Ruta del backend
backend\src\database-connection-verify.d.ts	Ruta del backend
backend\src\index.ts	Ruta del backend
backend\src\lib\territorios.ts	Ruta del backend
backend\src\middleware\auth.ts	Ruta del backend
backend\src\middleware\errorHandler.ts	Ruta del backend
backend\src\middleware\upload.ts	Ruta del backend
backend\src\middleware\validation.ts	Ruta del backend
backend\src\routes\admin.ts	Ruta del backend
backend\src\routes\anuncios.ts	Ruta del backend
backend\src\routes\auth.ts	Ruta del backend
backend\src\routes\comunidad.ts	Ruta del backend
backend\src\routes\estadisticas.ts	Ruta del backend
backend\src\routes\eventos.ts	Ruta del backend
backend\src\routes\moderacion.ts	Ruta del backend
backend\src\routes\propuestas.ts	Ruta del backend
backend\src\routes\provincias.ts	Ruta del backend
backend\src\routes\recursos.ts	Ruta del backend
backend\src\routes\reportes.ts	Ruta del backend
backend\src\routes\sugerencias.ts	Ruta del backend
backend\src\routes\territorios.ts	Ruta del backend
backend\src\routes\upload.ts	Ruta del backend
backend\src\routes\usuarios.ts	Ruta del backend
backend\src\types\express.d.ts	Ruta del backend
backend\src\utils\audit.ts	Ruta del backend
backend\src\utils\categorias.ts	Ruta del backend
backend\src\utils\ip.ts	Ruta del backend
backend\src\utils\logger.ts	Ruta del backend

7.12. Endpoints completos de la API

La tabla siguiente recoge los endpoints detectados en el código del backend. Se han identificado a partir de las definiciones de router en backend/src.

Método

Método	Ruta	Módulo
--------	------	--------

APP	/API/HEALTH	Ruta de la API
APP	/API/TEST-DB	Ruta de la API
APP	/HEALTH	Ruta de la API
APP	/TEST-DB	Ruta de la API
ROUTER	/:COMUNIDAD	Ruta de la API
ROUTER	/:ID	Ruta de la API
ROUTER	/:ID/APOYAR	Ruta de la API
ROUTER	/:ID/GUARDAR	Ruta de la API
ROUTER	/:ID/MODERAR	Ruta de la API
ROUTER	/:ID/MODERATE	Ruta de la API
ROUTER	/:ID/PERFIL	Ruta de la API
ROUTER	/:ID/REPORTAR	Ruta de la API
ROUTER	/:ID/REPORTES	Ruta de la API
ROUTER	/AGENDA	Ruta de la API
ROUTER	/AGENDA/:ID	Ruta de la API
ROUTER	/ANUNCIOS	Ruta de la API
ROUTER	/COMUNICACIONES	Ruta de la API
ROUTER	/COMUNICACIONES/:ID	Ruta de la API
ROUTER	/COMUNICACIONES/:ID/ENTIDADES	Ruta de la API
ROUTER	/COMUNICACIONES/:ID/ENVIADO	Ruta de la API
ROUTER	/COMUNICACIONES/:ID/EXPORTAR	Ruta de la API
ROUTER	/COMUNIDAD	Ruta de la API
ROUTER	/COMUNIDAD/PUBLICACIONES/:ID/ESTADO	Ruta de la API
ROUTER	/COMUNIDAD/REPORTES	Ruta de la API
ROUTER	/COMUNIDAD/REPORTES/:ID/REVISAR	Ruta de la API
ROUTER	/COMUNIDAD/RESPUESTAS	Ruta de la API
ROUTER	/COMUNIDAD/RESPUESTAS/:ID/ESTADO	Ruta de la API
ROUTER	/COMUNIDADES	Ruta de la API
ROUTER	/CONTACTOS-INSTITUCIONALES	Ruta de la API
ROUTER	/CONTACTOS-INSTITUCIONALES/:ID	Ruta de la API
ROUTER	/CONTACTOS-INSTITUCIONALES/:ID/VERIFICAR	Ruta de la API
ROUTER	/DASHBOARD	Ruta de la API
ROUTER	/DESTINATARIOS/BUSCAR	Ruta de la API
ROUTER	/ESTADISTICAS	Ruta de la API

ROUTER	/FORGOT-PASSWORD	Ruta de la API
ROUTER	/GUARDADOS	Ruta de la API
ROUTER	/HOME	Ruta de la API
ROUTER	/IMAGEN	Ruta de la API
ROUTER	/LOGIN	Ruta de la API
ROUTER	/LOGOUT	Ruta de la API
ROUTER	/LOGS	Ruta de la API
ROUTER	/ME	Ruta de la API
ROUTER	/MENSAJES	Ruta de la API
ROUTER	/MENSAJES/:ID	Ruta de la API
ROUTER	/MENSAJES/:ID/ADJUNTOS	Ruta de la API
ROUTER	/MENSAJES/:ID/ARCHIVAR	Ruta de la API
ROUTER	/MENSAJES/:ID/LEIDO	Ruta de la API
ROUTER	/MENSAJES/:ID/RESPONDER	Ruta de la API
ROUTER	/MENSAJES/ADJUNTOS/:ADJUNTOID/DESCARGAR	Ruta de la API
ROUTER	/MENSAJES/NO-LEIDOS	Ruta de la API
ROUTER	/MENSAJES/STAFF	Ruta de la API
ROUTER	/MIS-ANUNCIOS	Ruta de la API
ROUTER	/MODERACION	Ruta de la API
ROUTER	/MODERADORES	Ruta de la API
ROUTER	/MODERAR-BULK	Ruta de la API
ROUTER	/NECESIDADES	Ruta de la API
ROUTER	/NECESIDADES-JUVENILES	Ruta de la API
ROUTER	/PLANTILLAS	Ruta de la API
ROUTER	/PLANTILLAS/:ID	Ruta de la API
ROUTER	/PLANTILLAS/:ID/GENERAR-BORRADOR	Ruta de la API
ROUTER	/PROFILE	Ruta de la API
ROUTER	/PROVINCIA/:PROVINCIA	Ruta de la API
ROUTER	/PROVINCIAS	Ruta de la API
ROUTER	/PUBLICACION/:ID	Ruta de la API
ROUTER	/PUBLICACIONES/:ID/LIKE	Ruta de la API
ROUTER	/PUBLICACIONES/:ID/REPORTAR	Ruta de la API
ROUTER	/PUBLICACIONES/:ID/RESPUESTAS	Ruta de la API
ROUTER	/REFRESH	Ruta de la API

ROUTER	/REGISTER	Ruta de la API
ROUTER	/RESET-PASSWORD	Ruta de la API
ROUTER	/RESPUESTAS/:ID/LIKE	Ruta de la API
ROUTER	/RESPUESTAS/:ID/REPORTAR	Ruta de la API
ROUTER	/RESUMEN	Ruta de la API
ROUTER	/TAREAS	Ruta de la API
ROUTER	/TAREAS/:ID	Ruta de la API
ROUTER	/TAREAS/:ID/ESTADO	Ruta de la API
ROUTER	/TEMAS	Ruta de la API
ROUTER	/TERRITORIOS	Ruta de la API
ROUTER	/USUARIOS	Ruta de la API
ROUTER	/USUARIOS/:ID	Ruta de la API

7.13. Tablas de la base de datos

El esquema citypaj contiene las siguientes tablas, extraídas del script database/init.sql.

Tabla

Tabla	Finalidad
admin_activity_logs	Tabla del esquema citypaj
admin_tareas	Tabla del esquema citypaj
agenda_notas	Tabla del esquema citypaj
anuncios	Tabla del esquema citypaj
anuncios_guardados	Tabla del esquema citypaj
comunicaciones_adjuntos	Tabla del esquema citypaj
comunicaciones_entidades	Tabla del esquema citypaj
comunicaciones_institucionales	Tabla del esquema citypaj
comunidad_comentarios	Tabla del esquema citypaj
comunidad_likes	Tabla del esquema citypaj
comunidad_publicaciones	Tabla del esquema citypaj
comunidad_reportes	Tabla del esquema citypaj
comunidades	Tabla del esquema citypaj
contactos_institucionales	Tabla del esquema citypaj
eventos	Tabla del esquema citypaj
favoritos	Tabla del esquema citypaj
mensajes_adjuntos	Tabla del esquema citypaj
mensajes_entidades_adjuntas	Tabla del esquema citypaj

mensajes_staff	Tabla del esquema citypaj
moderacion_logs	Tabla del esquema citypaj
plantillas_comunicacion	Tabla del esquema citypaj
propuestas	Tabla del esquema citypaj
propuestas_apoyos	Tabla del esquema citypaj
provincias	Tabla del esquema citypaj
recursos	Tabla del esquema citypaj
reportes_anuncios	Tabla del esquema citypaj
sugerencias	Tabla del esquema citypaj
usuarios	Tabla del esquema citypaj

7.14. Diccionario de datos

A continuación se resume la finalidad de cada tabla principal.

Tabla

Tabla	Descripción
admin_activity_logs	Registro de actividades de administración.
admin_tareas	Tareas asignadas en el panel de administración.
agenda_notas	Notas y eventos de la agenda institucional.
anuncios	Publicaciones principales del sistema.
anuncios_guardados	Relación de anuncios guardados por usuarios.
comunicaciones_adjuntos	Archivos adjuntos a comunicaciones.
comunicaciones_entidades	Destinatarios de comunicaciones institucionales.
comunicaciones_institucionales	Borradores y envíos a entidades.
comunidad_comentarios	Comentarios de publicaciones de comunidad.
comunidad_likes	Me gusta de publicaciones y comentarios.
comunidad_publicaciones	Publicaciones del módulo de comunidad.
comunidad_reportes	Reportes de contenido de comunidad.
comunidades	Catálogo de comunidades autónomas.
contactos_institucionales	Contactos de entidades institucionales.
eventos	Eventos culturales y actividades.
favoritos	Anuncios marcados como favoritos.
mensajes_adjuntos	Adjuntos de mensajes del staff.
mensajes_entidades_adjuntas	Entidades vinculadas a mensajes.
mensajes_staff	Mensajes internos del staff.
moderacion_logs	Registro de acciones de moderación.

plantillas_comunicacion	Plantillas para comunicaciones.
propuestas	Iniciativas ciudadanas.
propuestas_apoyos	Apoyos recibidos por cada propuesta.
provincias	Catálogo de provincias vinculadas a comunidades.
recursos	Recursos juveniles y convocatorias.
reportes_anuncios	Reportes de anuncios.
sugerencias	Sugerencias enviadas por usuarios.
usuarios	Usuarios registrados del sistema.

7.15. Historial de commits relevantes

La tabla incluye los commits del repositorio, limpiando mensajes que contenían referencias no aptas para la memoria académica.

Hash

Hash	Fecha	Mensaje
329ef1a	2026-08-10	readme definitivo
39b908b	2026-08-10	Elimina scripts de prueba y archivos temporales
a66cdfb	2026-08-10	Permite publicar anuncios con todas las categorías y adjuntar carteles
5a00d2e	2026-08-10	Excluye anuncios culturales del home y anade utilidad de categorías
2e21e98	2026-08-10	Configura proxy interno del frontend hacia el backend en Docker
ebdab2b	2026-08-10	Implementa API de anuncios, filtro de cultura y utilidad de categorías
4362e19	2026-08-10	Configura Express, seguridad, conexion a MySQL y subida de imagenes
b38ea41	2026-08-10	Anade esquema y datos iniciales de MySQL
c737001	2026-08-10	Configura Docker, variables de entorno y scripts de build
d1eb22e	2026-08-09	Ajusta configuracion de despliegue del proyecto
d9efa04	2026-08-09	Fusionar historial
62fc736	2026-08-09	Actualizar Node a 24.x para
9996589	2026-08-09	Fijar version de Node a 20.x
710759f	2026-08-09	Trigger deploy
d7380ae	2026-08-05	AÑadir .json raÑ-z para build del frontend desde root

64d903d	2026-08-05	Initial commit
5245948	2026-08-05	Preparar next.config.js para dominio www.citypaj.es
70085a7	2026-08-05	Añadir configuración para despliegue del frontend
c4f8f6a	2026-08-05	Merge limpieza interna
45ee043	2026-08-05	Más limpieza
5ba506f	2026-08-05	Añadir islas, estilizar selects y actualizar README
668a8c4	2026-08-05	Merge correccion territorial
84a50d5	2026-08-05	Correccion territorial comunidades y provincias
c381459	2026-08-05	Correccion carga de estilos citypaj
d9d813c	2026-08-04	Merge limpieza en main
ffb7209	2026-08-04	Optimización de archivos del repositorio
647c0f0	2026-08-04	Limpieza final del repositorio para entrega
27d99c9	2026-08-04	Quitar footer en paneles de administracion
34e6c94	2026-08-04	Merge rama citypaj
b42bb51	2026-08-04	Limpieza proyecto
9356ed7	2026-08-04	Hacer botones de listado y favicon funcionales
50737c2	2026-08-04	Revisión
0c704cc	2026-08-04	Hacer guardar/reportar/compartir accesibles sin cuenta
e958ec0	2026-08-04	Actualizar términos/privacidad (IP), mejorar estética de instituciones, reescribir README y limpieza de referencias obsoletas
5bd878c	2026-08-02	Corregir tipado en calendario y revisión de carga
a12f420	2026-08-02	Añadir ilustración SVG a la sección de comunidad
9eb86a7	2026-08-02	Mejorar hovers en cultura
fc59b2d	2026-08-02	Ajustes cultura, publicar integrado, adjuntos, calendarios editables e ilustraciones
ca09df4	2026-08-02	Añadir notificaciones de mensajes, eventos culturales por provincia y datos

		mock de moderaci3n
8194b10	2026-08-02	Datos mock, emails, calendario y mensajer3a real para el panel admin
0d74434	2026-08-02	Admin Cultura, nav Cultura al final, provincia vinculada a CCAA en comunidad
3f890b7	2026-08-02	Eventos y anuncios culturales: mocks, cartel_url
4b6f589	2026-08-02	Nueva columna en anuncios, comunidad, propuestas, sugerencias y eventos
52a04e3	2026-08-02	Agenda/calendario para admin y moderador, plantillas visibles para moderadores, ajuste rate limit y campos quitados en comunicaci3n
61052ac	2026-08-02	icono Nuevo en segunda posici3n en mensajer3a, recarga de plantillas con datos mocks de plantillas en BD
e7564c1	2026-08-02	Arreglo las plantillas: el select de comunicaciones solo muestra las de tipo institucional con asunto y cuerpo rellenos, y el backend ya devuelve el cuerpo
c109a18	2026-08-02	Implemento la refactorizaci3n del panel admin y mensajer3a: destinatarios inteligentes, agenda institucional, plantillas (de lo que me ha podido decir), comunicaciones, tareas y dashboard de necesidades juveniles --- no deja de ser una demo
6e2c9b5	2026-08-02	diferenciaci3n de roles, mensajer3a interna con adjuntos (:D!!!), logs y estad3sticas
120b4f0	2026-08-01	panel admin semicompleto (pendiente completar), mensajer3a interna, favicon y login por rol
0edc19b	2026-07-31	A3ado hover naranja en cards de ayudas y b3squeda y ajusto color de enlaces
3e8e4ad	2026-07-31	instrucciones de despliegue con Docker y datos reales
63b37c5	2026-07-31	docker compose modificando seguridad
890a4ad	2026-07-31	ajuste de Docker para despliegue
4a03e93	2026-07-31	limpieza de referencias obsoletas

11be5a1	2026-07-31	ilustración svg para hero
3d9b106	2026-07-31	rediseño de la página de instituciones de cara a la venta
f5c66cb	2026-07-30	elimino rutas y páginas antiguas de comunidad
f1c7d56	2026-07-30	páginas de detalle, creación, provincia y moderación
e383ec3	2026-07-30	componentes y página principal de comunidad
9a5f669	2026-07-30	endpoints y moderación de comunidad
e072843	2026-07-30	datos de ejemplo de comunidad
f174492	2026-07-30	permite participación anónima con nombre e ip
4f6b488	2026-07-30	migración base para likes y reportes de comunidad
a023229	2026-07-29	limpieza: borro temporales, console.log y actualizo README
0e65020	2026-07-29	scripts pa revisar la base de datos
8dc9575	2026-07-29	actualizo package.json del backend
b08e2e2	2026-07-29	retoques home, filtros y provincias
24cb96e	2026-07-29	panel admin: filtros, aplicar, limpiar y responsive
6e6c85f	2026-07-29	arreglo busqueda y reportes en moderacion
a206a2b	2026-07-29	nuevo endpoint /api/admin/anuncios pa los filtros del panel
f5ce62e	2026-07-24	en el panel de moderación: filtros, orden, paginacion, acciones masivas, plantillas de rechazo, preview inline, auditoria y export CSV
5a8139a	2026-07-24	chore: eliminar log de runtime del control de versiones
b313598	2026-07-24	docs: README completo con montaje, comandos, arquitectura y moderacion
75c9d06	2026-07-24	fix: moderacion con estado, notas y acciones; acceso desde footer a login moderador
622444e	2026-07-24	filtros CCAA/provincia en home, logout centrado y panel moderación responsive

b4dc693	2026-07-23	ui: linea fina debajo del hero
1a21ee8	2026-07-23	titulo hero negro y rejected solo con revision humana
5960ee5	2026-07-23	buscador nav funcional, enlace INAEM/empleo en ayudas y favicon SVG
a7115dd	2026-07-23	ui: titulo hero en negrita y secciones del nav con letra mas grande
c81eebd	2026-07-23	ui: hero nuevo con colores vivos, modal de logout centrado y acceso moderador más discreto
8584fca	2026-07-23	eliminado sistema de traducción
a336ecf	2026-07-23	ui: hover de enlaces distinto al color base
728221b	2026-07-23	scripts limpieza puertos, predev, check:ports y README actualizado
bdf6ae5	2026-07-23	del front: acceso admin, panel moderador, api translate y modal reporte
d679d99	2026-07-23	del front: ajustes paginas usuario, admin, anuncios y componentes Footer/ListingRow
e15537e	2026-07-23	del back: endpoints cuenta/perfil, una demo seed, migration rol y mejoras en los controllers
5bdd4a1	2026-07-23	fix(backend): tipado requestId, tsconfig dev y bind 0.0.0.1
67cd80f	2026-07-23	Ajustes de filtros, provincias, nav movil, traduccion pagina completa y correccion demo login
5bb5e6f	2026-07-23	correcciones
b05c4f6	2026-07-23	nav arreglado
473ec85	2026-07-23	selector de idioma con
73d935b	2026-07-23	selector de idioma con (pendiente finalizar)
2873205	2026-07-23	fix: paginacion
42cb6cb	2026-07-23	quitar busqueda del hero
81165b3	2026-07-23	nav persistente y buscador funcional
2f9cd7c	2026-07-23	feat: publicar anuncios sin cuenta
8206103	2026-07-22	otros cambios
ffd77f4	2026-07-22	listados y home de anuncios
442f230	2026-07-22	comunidad y foros por ccaa

2fea210	2026-07-22	auth y sesiones
d547d80	2026-07-22	fix idiomas todav�a pendiente
b0137df	2026-07-22	panel de moderacion de anuncios
bdd364d	2026-07-22	publicar anuncio con revision
497b0c4	2026-07-22	navegacion y pagina de ayudas
bc25e4e	2026-07-22	actualizado README
6d8cf63	2026-06-16	Limpieza y optimizaci�n completa del proyecto
b829188	2026-06-15	Reconducci�n proyecto parte 1 - Conexi�n con bbdd, pendiente reorganizar
179e3b1	2026-06-15	���� CityPaj TFG - Versi�n Final Completa
405b1a3	2026-06-01	Creo la p�gina de recuperar contrase�a
95049d9	2026-06-01	Elimino la fila de comunidades del grid principal
3c179c1	2026-06-01	A��ado la imagen fondo-hero1 al hero
4fb98a2	2026-06-01	Recupero la l�gica anterior de traducci�n
64a9420	2026-05-28	Proyecto CityPaj CASI completo y funcional
3547ac4	2026-05-28	correcciones TypeScript y rutas API - URLSearchParams: construcci�n correcta de par�metros
6182ab2	2026-05-28	limpieza total de simulaciones
d69b9e4	2026-05-28	backend adaptado a base de datos MySQL real
ffbab68	2026-05-28	a��adido sistema de filtrado avanzado en el frontend
e8d5816	2026-05-28	Resolver conflictos de merge y sincronizar cambios
078a3d3	2026-05-28	Implementar sistema de anuncios guardados funcionando
96f3226	2026-05-28	Implementar sistema de anuncios guardados funcionando

7.16. Plan de pruebas detallado

El plan de pruebas ampl a el incluido en el apartado 4.3.2 con casos concretos y estados reales del entorno de generaci n.

Código

Código	Prueba	Pasos	Esperado	Obtenido	Estado
TP-01	Prueba de registro	Acceder al formulario, rellenar datos y enviar.	Cuenta creada en base de datos.	No ejecutada	No ejecutada
TP-02	Prueba de login	Introducir credenciales de demostración.	Token JWT devuelto.	No ejecutada	No ejecutada
TP-03	Prueba de logout	Pulsar cerrar sesión.	Sesión eliminada en cliente.	No ejecutada	No ejecutada
TP-04	Consulta de anuncios	Cargar el home.	Listado paginado visible.	No ejecutada	No ejecutada
TP-05	Detalle de anuncio	Pulsar un anuncio.	Página con información completa.	No ejecutada	No ejecutada
TP-06	Filtro por categoría	Seleccionar categoría en barra de filtros.	Listado filtrado.	No ejecutada	No ejecutada
TP-07	Filtro por CCAA	Seleccionar comunidad autónoma.	Provincias correspondientes cargadas.	No ejecutada	No ejecutada
TP-08	Filtro por provincia	Seleccionar provincia.	Anuncios filtrados por territorio.	No ejecutada	No ejecutada
TP-09	Publicación de anuncio	Rellenar formulario y enviar.	Anuncio en estado pending.	No ejecutada	No ejecutada
TP-10	Edición de anuncio propio	Editar anuncio desde el perfil.	Datos actualizados.	No ejecutada	No ejecutada
TP-11	Guardar favorito	Pulsar icono de favorito.	Anuncio en lista de guardados.	No ejecutada	No ejecutada
TP-12	Reportar anuncio	Enviar reporte.	Registro en reportes_anuncios.	No ejecutada	No ejecutada
TP-13	Publicación en comunidad	Crear publicación anónima.	Publicación visible con IP.	No ejecutada	No ejecutada
TP-14	Comentar publicación	Añadir comentario.	Comentario en comunidad_comentarios.	No ejecutada	No ejecutada
TP-15	Enviar sugerencia	Rellenar buzón de sugerencias.	Registro en sugerencias.	No ejecutada	No ejecutada
TP-16	Apoyar propuesta	Pulsar apoyo en propuesta.	Registro en propuestas_apoyos.	No ejecutada	No ejecutada
TP-17	Consultar recursos	Acceder a página de recursos.	Listado visible.	No ejecutada	No ejecutada

TP-18	Consultar cultura y eventos	Acceder a sección cultura.	Eventos visibles.	No ejecutada	No ejecutada
TP-19	Panel de moderación	Acceder con rol moderador.	Cola de revisión accesible.	No ejecutada	No ejecutada
TP-20	Panel de administración	Acceder con rol admin.	Dashboard y módulos visibles.	No ejecutada	No ejecutada
TP-21	Prueba de API /health	Llamar a GET /health.	Respuesta 200 con estado.	No ejecutada	No ejecutada
TP-22	Prueba de API anuncios	Llamar a GET /api/anuncios.	Listado JSON de anuncios.	No ejecutada	No ejecutada
TP-23	Prueba de base de datos	Conectar cliente MySQL.	Tablas y datos presentes.	No ejecutada	No ejecutada
TP-24	Prueba de Docker	Ejecutar docker compose up --build.	Tres servicios saludables.	No ejecutada	No ejecutada
TP-25	Prueba de seguridad básica	Acceder a ruta admin sin token.	Respuesta 401 o 403.	No ejecutada	No ejecutada
TP-26	Prueba de inyección SQL	Enviar comilla simple en formulario.	Sin error de base de datos.	No ejecutada	No ejecutada
TP-27	Prueba responsive	Redimensionar a 375 px.	Menú y grillas ajustadas.	No ejecutada	No ejecutada
TP-28	Prueba de errores controlados	Enviar datos inválidos.	Mensaje de error claro.	No ejecutada	No ejecutada

7.17. Lista de comprobación final

Antes de la entrega se ha revisado la memoria según los siguientes criterios.

Criterio

Criterio	Cumplimiento
Portada completa con datos del proyecto	Sí
Índice general, de tablas y de ilustraciones	Sí
Estructura académica completa de 5 documentos + anexos	Sí
Repositorio https://github.com/carmendmv/citypaj	Sí
Base de datos citypaj con MySQL 8.0	Sí
Frontend en http://localhost:3001	Sí
Backend en http://localhost:3002	Sí
14 diagramas SVG generados	Sí
Tecnologías reales extraídas del repositorio	Sí
Tablas y endpoints reales del repositorio	Sí

Historial de commits incluido	Sí
Emoticonos eliminados	Sí
Términos prohibidos eliminados	Sí
Rastros de herramientas internas eliminados	Sí
Sin referencias a repositorios obsoletos	Sí

7.18. Variables de entorno de ejemplo

A continuación se incluyen los archivos de ejemplo de variables de entorno del proyecto.

Archivo `.env.example` de la raíz:

```
# Configuración general de CityPAJ para Docker local
NODE_ENV=development
PORT=3001

# MySQL
DB_HOST=mysql
DB_PORT=3306
DB_NAME=citypaj
DB_USER=citypaj_user
DB_PASSWORD=citypaj_password

# Backend
BACKEND_URL=http://localhost:3002
NEXT_PUBLIC_API_URL=http://localhost:3002/api
NEXT_PUBLIC_APP_URL=http://localhost:3001

# JWT (seguro únicamente para demo)
JWT_SECRET=citypaj_secret_demo
JWT_REFRESH_SECRET=citypaj_refresh_demo

# CORS
CORS_ORIGINS=http://localhost:3001

# Usuarios demo
DEMO_ADMIN_EMAIL=admin@citypaj.local
DEMO_ADMIN_PASSWORD=Admin1234
DEMO_MODERATOR_EMAIL=moderador@citypaj.local
DEMO_MODERATOR_PASSWORD=Moderador1234
DEMO_USUARIO_EMAIL=usuario@citypaj.local
DEMO_USUARIO_PASSWORD=Usuario1234
```

Archivo `backend/.env.example`:

```
# Servidor
PORT=3002
NODE_ENV=development

# MySQL (Docker)
DB_HOST=mysql
DB_PORT=3306
DB_NAME=citypaj
DB_USER=citypaj_user
DB_PASSWORD=citypaj_password

# JWT (seguro únicamente para demo)
JWT_SECRET=citypaj_secret_demo
```

```
JWT_REFRESH_SECRET=citypaj_refresh_demo
```

```
#
CORS_ORIGINS=http://localhost:3001
```

```
#
DEMO_ADMIN_EMAIL=admin@citypaj.local
DEMO_ADMIN_PASSWORD=Admin1234
DEMO_MODERATOR_EMAIL=moderador@citypaj.local
DEMO_MODERATOR_PASSWORD=Moderador1234
DEMO_USUARIO_EMAIL=usuario@citypaj.local
DEMO_USUARIO_PASSWORD=Usuario1234
```

7.19. Docker Compose

El archivo `docker-compose.yml` del repositorio contiene la orquestación de los tres servicios. A continuación se incluye el contenido completo del archivo.

```
services:
  mysql:
    image: mysql:8.0
    container_name: citypaj-mysql
    environment:
      MYSQL_DATABASE: citypaj
      MYSQL_USER: citypaj_user
      MYSQL_PASSWORD: citypaj_password
      MYSQL_ROOT_PASSWORD: citypaj_root
      MYSQL_ALLOW_EMPTY_ROOT_PASSWORD: "no"
    volumes:
      - citypaj_mysql_data:/var/lib/mysql
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "mysql -u citypaj_user -pcitypaj_password -D citypaj -e 'SELECT 1 FROM usuarios LIMIT 1' > /dev/null 2>&1 || exit 1"]
      interval: 10s
      timeout: 5s
      retries: 10
      start_period: 60s
      restart: unless-stopped

  backend:
    build:
      context: ./backend
      target: production
    container_name: citypaj-backend
    ports:
      - "3002:3002"
    environment:
      NODE_ENV: development
      PORT: 3002
      DB_HOST: mysql
      DB_PORT: 3306
      DB_NAME: citypaj
      DB_USER: citypaj_user
      DB_PASSWORD: citypaj_password
      JWT_SECRET: citypaj_secret_demo
      JWT_REFRESH_SECRET: citypaj_refresh_demo
      CORS_ORIGINS: http://localhost:3001
      DEMO_ADMIN_EMAIL: admin@citypaj.local
      DEMO_ADMIN_PASSWORD: Admin1234
```

```

    DEMO_MODERATOR_EMAIL:          moderador@citypaj.local
    DEMO_MODERATOR_PASSWORD:      Moderador1234
    DEMO_USUARIO_EMAIL:           usuario@citypaj.local
    DEMO_USUARIO_PASSWORD:        Usuario1234
  depends_on:
    mysql:
      condition:                   service_healthy
  restart:                         on-failure:5

frontend:
  build:
    context:                       ../frontend
  container_name:                  citypaj-frontend
  ports:
    -                              "3001:3001"
  environment:
    NODE_ENV:                      production
    BACKEND_URL:                   http://backend:3002
    NEXT_PUBLIC_API_URL:           http://localhost:3002/api
    API_URL_DOCKER:                http://backend:3002
    NEXT_PUBLIC_APP_URL:           http://localhost:3001
  depends_on:
    -                              backend
  restart:                         on-failure:5

volumes:
  citypaj_mysql_data:
    driver:                        local

```

7.20. Extracto del script de base de datos

A continuación se incluyen las primeras líneas del script database/init.sql como referencia documental. El archivo completo se encuentra en el repositorio.

```

USE                                                                    citypaj;

--      MySQL      dump      10.13      Distrib      8.0.40,      for      Linux      (x86_64)
--
--      Host:      host.docker.internal      Database:      citypaj
--
--      Server      version      5.5.5-10.4.32-MariaDB

/*!40101      SET      @OLD_CHARACTER_SET_CLIENT=@@CHARACTER_SET_CLIENT      */;
/*!40101      SET      @OLD_CHARACTER_SET_RESULTS=@@CHARACTER_SET_RESULTS      */;
/*!40101      SET      @OLD_COLLATION_CONNECTION=@@COLLATION_CONNECTION      */;
/*!50503      SET      NAMES      utf8mb4      */;
/*!40103      SET      @OLD_TIME_ZONE=@@TIME_ZONE      */;
/*!40103      SET      TIME_ZONE='+00:00'      */;
/*!40014      SET      @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS,      UNIQUE_CHECKS=0      */;
/*!40014      SET      @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS,      FOREIGN_KEY_CHECKS=0      */;
/*!40101      SET      @OLD_SQL_MODE=@@SQL_MODE,      SQL_MODE='NO_AUTO_VALUE_ON_ZERO'      */;
/*!40111      SET      @OLD_SQL_NOTES=@@SQL_NOTES,      SQL_NOTES=0      */;

--
--      Table      structure      for      table      `admin_activity_logs`
--

DROP      TABLE      IF      EXISTS      `admin_activity_logs`;
/*!40101      SET      @saved_cs_client      =      @@character_set_client      */;
/*!50503      SET      character_set_client      =      utf8mb4      */;

```

```

CREATE TABLE `admin_activity_logs` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `usuario_id` varchar(36) DEFAULT NULL,
  `accion` varchar(100) NOT NULL,
  `entidad` varchar(100) DEFAULT NULL,
  `entidad_id` varchar(255) DEFAULT NULL,
  `detalles` text DEFAULT NULL,
  `ip` varchar(64) DEFAULT NULL,
  `creado_at` timestamp NOT NULL DEFAULT current_timestamp(),
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=21 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
/*!40101 SET character_set_client = @saved_cs_client */;

```

```

--
-- Dumping data for table `admin_activity_logs`
--

```

```

LOCK TABLES `admin_activity_logs` WRITE;
/*!40000 ALTER TABLE `admin_activity_logs` DISABLE KEYS */;
INSERT INTO `admin_activity_logs` VALUES (1,'305eff30-ea27-4511-a2a8-774b236d22c8','inicio_sesion','sesion',NULL,'Inicio de sesión en el panel','127.0.0.1','2026-08-02 14:03:33'),(2,'b99c51b6-756d-4c18-98c8-a1d479ad6224','moderar_anuncio','anuncio',NULL,'Aprobó un anuncio de empleo','127.0.0.1','2026-08-02 14:03:33'),(3,'332610e5-89e8-49c6-abdc-81242db69c79','crear_tarea','tarea',NULL,'Creó una tarea de verificación','127.0.0.1','2026-08-02 14:03:33'),(4,'305eff30-ea27-4511-a2a8-774b236d22c8','enviar_comunicacion','comunicacion',NULL,'Envío comunicación a institución','127.0.0.1','2026-08-02 14:03:33'),(5,'b99c51b6-756d-4c18-98c8-a1d479ad6224','responder_mensaje','mensaje',NULL,'Respondió mensaje de staff','127.0.0.1','2026-08-02 14:03:33'),(6,'305eff30-ea27-4511-a2a8-774b236d22c8','moderar_anuncio','anuncio','d33302de-48a2-4e68-88e9-0e48f0b52924','Aprobó anuncio','127.0.0.1','2026-08-02 12:43:47'),(7,'b99c51b6-756d-4c18-98c8-a1d479ad6224','rechazar_anuncio','anuncio','0dc2f361-2ff8-443a-ab9f-14eab58c8881','Rechazó anuncio','127.0.0.1','2026-08-01 12:43:47'),(8,'332610e5-89e8-49c6-abdc-81242db69c79','revisar_reporte','reporte','aded8915-d06c-476f-9e14-52476d12304d','Revisó reporte de comunidad','127.0.0.1','2026-07-31 12:43:47'),(9,'305eff30-ea27-4511-a2a8-774b236d22c8','moderar_publicacion','publicacion','87b259b8-ad0d-412b-9896-f1aacd834f3f','Aprobó publicación','127.0.0.1','2026-07-30 12:43:47'),(10,'b99c51b6-756d-4c18-98c8-a1d479ad6224','moderar_comentario','comentario','251052f6-d9a0-46af-80dd-a7f9c52d7a5f','Ocultó comentario','127.0.0.1','2026-07-29 12:43:47'),(11,'332610e5-89e8-49c6-abdc-81242db69c79','resolver_sugerencia','sugerencia','8fc5b295-fbe5-4a7c-af5b-951cd8852b32','Marcó sugerencia en progreso','127.0.0.1','2026-07-28 12:43:47'),(12,'305eff30-ea27-4511-a2a8-774b236d22c8','moderar_propuesta','propuesta','e6b83e29-03c7-4b0c-bf87-47b176e3426e','Aprobó propuesta','127.0.0.1','2026-07-27 12:43:47'),(13,'b99c51b6-756d-4c18-98c8-a1d479ad6224','moderar_anuncio','anuncio','91a142bc-c706-488a-8048-a888856f2ab','Aprobó anuncio','127.0.0.1','2026-07-26 12:43:47'),(14,'332610e5-89e8-49c6-abdc-81242db69c79','rechazar_anuncio','anuncio','91c01841-11fd-4a55-a1d0-a469aa85e4ff','Rechazó anuncio','127.0.0.1','2026-07-25 12:43:47'),(15,'305eff30-ea27-4511-a2a8-774b236d22c8','revisar_reporte','reporte','12170a7a-7504-4629-b5fe-fc828bf89b58','Revisó reporte de comunidad','127.0.0.1','2026-07-24 12:43:47'),(16,'b99c51b6-756d-4c18-98c8-a1d479ad6224','moderar_publicacion','publicacion','726e5543-0fc9-44b0-b32f-a1c8d25b039d','Aprobó publicación','127.0.0.1','2026-07-23 12:43:47'),(17,'332610e5-89e8-49c6-abdc-81242db69c79','moderar_comentario','comentario','2679a75f-53ed-4437-982e-e7f0a1992508','Ocultó comentario','127.0.0.1','2026-07-22 12:43:47'),(18,'305eff30-ea27-4511-a2a8-774b236d22c8','resolver_sugerencia','sugerencia','c7e161c1-7c30-45d7-bb8f-23c36c497cc1','Marcó sugerencia en progreso','127.0.0.1','2026-07-21 12:43:47'),(19,'b99c51b6-756d-4c18-98c8-a1d479ad6224','moderar_propuesta','propuesta','be80bb24-a01c-43b7-bf6c-7c553de1ecc7','Aprobó propuesta','127.0.0.1','2026-07-20 12:43:47'),(20,'332610e5-89e8-49c6-abdc-81242db69c79','moderar_anuncio','anuncio','5bd67062-071a-4caa-9f7e-24d50717c86b','Aprobó anuncio','127.0.0.1','2026-07-19 12:43:47');
/*!40000 ALTER TABLE `admin_activity_logs` ENABLE KEYS */;
UNLOCK TABLES;

```

```

--
-- Table structure for table `admin_tareas`

```

```
--
DROP TABLE IF EXISTS `admin_tareas`;
/*!40101 SET @saved_cs_client = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `admin_tareas` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `titulo` varchar(255) NOT NULL,
  `descripcion` text DEFAULT NULL,
  `estado` enum('pendiente','en_progreso','completada','cancelada') DEFAULT 'pendiente',
  `prioridad` enum('baja','media','alta','urgente') DEFAULT 'media',
  `asignado_a` varchar(36) DEFAULT NULL,
  `creado_por` varchar(36) DEFAULT NULL,
  `entidad_tipo` varchar(50) DEFAULT NULL,
  `entidad_id` varchar(255) DEFAULT NULL,
  `vencimiento` datetime DEFAULT NULL,
  `creado_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `actualizado_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(),
  PRIMARY KEY (`id`)
) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
/*!40101 SET @saved_cs_client = @@character_set_client */;
```

```
--
-- Dumping data for table `admin_tareas`
--
LOCK TABLES `admin_tareas` WRITE;
/*!40000 ALTER TABLE `admin_tareas` DISABLE KEYS */;
INSERT INTO `admin_tareas` VALUES (1,'Revisar anuncios culturales','Moderar nuevos anuncios del apartado Cultura.','pendiente','media','b99c51b6-756d-4c18-98c8-a1d479ad6224','305eff30-ea27-4511-a2a8-774b236d22c8','anuncio',NULL,'2026-08-10 00:00:00','2026-08-02 14:03:33','2026-08-02 14:03:33'),(2,'Contactar ayuntamiento Zaragoza','Trasladar sugerencias de vivienda juvenil.','en_progreso','alta','332610e5-89e8-49c6-abdc-81242db69c79','305eff30-ea27-4511-a2a8-774b236d22c8','sugerencia',NULL,'2026-08-05 00:00:00','2026-08-02 14:03:33','2026-08-02 14:03:33'),(3,'Actualizar calendario de eventos','Añadir fechas de conciertos y talleres.','pendiente','baja','b99c51b6-756d-4c18-98c8-a1d479ad6224','305eff30-ea27-4511-a2a8-774b236d22c8','evento',NULL,'2026-08-20 00:00:00','2026-08-02 14:03:33','2026-08-02 14:03:33'),(4,'Verificar contacto institucional','Confirmar email del departamento de juventud.','completada','media','332610e5-89e8-49c6-abdc-81242db69c79','305eff30-ea27-4511-a2a8-774b236d22c8','contacto',NULL,'2026-07-30 00:00:00','2026-08-02 14:03:33','2026-08-02 14:03:33'),(5,'Responder mensaje de Pablo','Aclarar dudas sobre moderación.','pendiente','urgente','305eff30-ea27-4511-a2a8-774b236d22c8','b99c51b6-756d-4c18-98c8-a1d479ad6224','mensaje',NULL,'2026-08-02 00:00:00','2026-08-02 14:03:33','2026-08-02 14:03:33');
/*!40000 ALTER TABLE `admin_tareas` ENABLE KEYS */;
UNLOCK TABLES;
```

```
--
-- Table structure for table `agenda_notas`
--
DROP TABLE IF EXISTS `agenda_notas`;
/*!40101 SET @saved_cs_client = @@character_set_client */;
/*!50503 SET character_set_client = utf8mb4 */;
CREATE TABLE `agenda_notas` (
  `id` int(11) NOT NULL AUTO_INCREMENT,
  `titulo` varchar(255) NOT NULL,
  `cuerpo` text DEFAULT NULL,
  `fecha` date NOT NULL,
  `color` varchar(20) DEFAULT 'orange',
  `usuario_id` varchar(36) NOT NULL,
  `creado_at` timestamp NOT NULL DEFAULT current_timestamp(),
  `actualizado_at` timestamp NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(),
  PRIMARY KEY (`id`)
)
```

```

PRIMARY
) ENGINE=InnoDB AUTO_INCREMENT=5 DEFAULT KEY CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
/*!40101 SET character_set_client = @saved_cs_client */;

--
-- Dumping data for table `agenda_notas`
--

LOCK TABLES `agenda_notas` WRITE;
/*!40000 ALTER TABLE `agenda_notas` DISABLE KEYS */;
INSERT INTO `agenda_notas` VALUES (1,'Reunión semanal de coordinación','Punto de situación de moderación
y tareas.','2026-08-05','blue','305eff30-ea27-4511-a2a8-774b236d22c8','2026-08-02 14:03:33','2026-08-02
14:03:33'),(2,'Festival de Jazz','Concierto al aire libre.','2026-09-10','green','b99c51b6-756d-4c18-
98c8-a1d479ad6224','2026-08-02 14:03:33','2026-08-02 14:03:33'),(3,'Taller de Poesía','Encuentro
literario.','2026-09-18','purple','332610e5-89e8-49c6-abdc-81242db69c79','2026-08-02 14:03:33','2026-08-
02 14:03:33'),(4,'Cierre de evaluación mensual','Revisar métricas y necesidades juveniles.','2026-08-
31','red','305eff30-ea27-4511-a2a8-774b236d22c8','2026-08-02 14:03:33','2026-08-02 14:03:33');

```

7.21. README del proyecto

A continuación se incluyen las primeras líneas del archivo README.md como referencia documental. El archivo completo se encuentra en el repositorio.

```

# CityPAJ - Plataforma de anuncios y servicios juveniles
## 1. Qué es CityPAJ

```

Este proyecto, **CityPAJ**, es mi Trabajo de Fin de Grado del Grado Superior de Desarrollo de Aplicaciones Web. He construido una aplicación web fullstack dirigida a jóvenes para que puedan publicar anuncios, buscar servicios, formación, empleo, vivienda y otras oportunidades cercanas a su comunidad.

El objetivo era crear una plataforma útil y sencilla: un usuario puede publicar un anuncio, que pasa por revisión de un moderador antes de ser visible. También he incluido paneles de administración y moderación para gestionar esos contenidos.

```

## 2. Stack técnico y por qué lo elegí

```

A lo largo del ciclo fui aprendiendo diferentes tecnologías y quise ponerlas en práctica en un solo proyecto. La elección fue la siguiente:

- **Frontend**: Next.js 14 con React y TypeScript, para practicar enrutamiento, estado y tipado seguro.
- **Estilos**: Tailwind CSS, porque me ha permitido maquetar rápido y mantener consistencia visual.
- **Backend**: Node.js + Express + TypeScript, para entender bien el flujo de peticiones, middlewares y controladores.
- **Base de datos**: MySQL 8.0, donde he diseñado el esquema relacional y he trabajado consultas, filtros y transacciones.
- **Contenedores**: Docker y Docker Compose, porque al final del proyecto vi que dockerizarlo facilitaba mucho la entrega y la revisión.

```

## 3. Arquitectura del proyecto

```

La comunicación entre las partes es directa:

```

...

```

Usuario



Frontend

Next.js



http://localhost:3001



Backend

Express



http://localhost:3002



MySQL (Docker) → base de datos citypaj

El usuario interactúa con el frontend en Next.js. Cuando se necesitan datos, las rutas internas de `app/api` hacen de proxy y se comunican con el backend a través de la red de Docker. El backend ejecuta las consultas en MySQL y devuelve el resultado en JSON.

He entregado el proyecto ****100 % dockerizado****. Con un solo comando se levantan frontend, backend y base de datos, sin necesidad de instalar nada más ni depender de servicios externos como , Render o PlanetScale.

4. Requisitos previos

Solo se necesita:

- ****Docker**** en ejecución.
- ****Git**** para clonar el repositorio.

No hace falta tener Node.js o MySQL instalados, ya que toda la aplicación corre dentro de los contenedores.

5. Instalación y primer arranque

Clonar el repositorio y entrar en la carpeta:

```
```bash
git clone URL_DEL_REPOSITORIO
cd citypaj
```
```

Los valores por defecto están en `docker-compose.yml` y en los archivos `.env.example`, así que no es necesario configurar nada la primera vez.

Para levantar el proyecto:

```
```bash
docker compose up --build
```
```

La primera vez Docker descarga las imágenes, instala dependencias, compila el backend y el frontend, y ejecuta `database/init.sql` para crear la base de datos. Puede tardar varios minutos.

6. URLs de acceso

Una vez los contenedores estén saludables:

```
```text
Frontend: http://localhost:3001
Backend: http://localhost:3002
Health check: http://localhost:3002/health
```
```

El health check sirve para comprobar que el backend ha conectado correctamente con MySQL.

7. Credenciales de prueba

Para probar los distintos roles he preparado tres usuarios de demostración que se crean automáticamente al arrancar el backend si no existen:

| Rol | Email | Contraseña |
|-------|---------------------|------------|
| Admin | admin@citypaj.local | Admin1234 |

| Moderador | moderador@citypaj.local | Moderador1234 |
|-----------|-------------------------|---------------------|
| Usuario | usuario@citypaj.local | Usuario1234 |
| ## | 8. | Cómo |
| Para | detener | los |
| | | contenedores |
| | | sin |
| | | perder |
| | | los |
| | | datos: |
| ```bash | | |
| docker | compose | down |
| ``` | | |
| Para | borrar | también |
| | | el |
| | | volumen |
| | | de |
| | | MySQL |
| | | y |
| | | empezar |
| | | desde |
| | | cero: |
| ```bash | | |
| docker | compose | down |
| ``` | | -v |
| ## | 9. | Cómo |
| | | reiniciar |
| | | la |
| | | base |
| | | de |
| | | datos |
| Si | se | modifica |
| | | `database/init.sql` |
| | | y |
| | | se |
| | | quieren |
| | | aplicar |
| | | los |
| | | cambios: |
| ```bash | | |
| docker | compose | down |
| docker | compose | up |
| ``` | | --build |
| Esto | recrea | el |
| | volumen | y |
| | vuelve | a |
| | cargar | el |
| | esquema | y |
| | los | datos |
| | iniciales. | |

Anexos técnicos adicionales

E.1. Descripción técnica del frontend

El frontend de CityPAJ se construye con Next.js 14 en su modelo de App Router. Este modelo permite organizar las páginas en carpetas bajo `frontend/src/app`, donde cada carpeta representa una ruta y el archivo `page.tsx` define el contenido.

Next.js ofrece renderizado del lado del servidor y generación estática. En este proyecto, las páginas se renderizan de forma dinámica en el servidor del contenedor, lo que permite acceder a variables de entorno y realizar peticiones al backend antes de servir el HTML.

La gestión de estado se realiza mediante Zustand para preferencias globales y React Context para la autenticación. Zustand es una biblioteca ligera que evita la verbosidad de Redux mientras mantiene un estado centralizado.

Los hooks personalizados encapsulan la lógica de acceso a la API. Por ejemplo, los hooks de anuncios gestionan la carga de listados, filtros y paginación. Los hooks de comunidad gestionan la consulta de publicaciones y comentarios.

Los formularios utilizan `react-hook-form` y `Zod`. `React-hook-form` controla el estado de los campos y reduce renderizados innecesarios. `Zod` valida los esquemas con tipos seguros.

La interfaz se compone de componentes reutilizables agrupados por área. Los componentes de ui incluyen botones, tarjetas, modales y barras de progreso. Los componentes de anuncios incluyen `AnuncioList`, `AnuncioCard` y `FiltroAvanzado`.

Tailwind CSS se configura en `frontend/tailwind.config.js`. Las clases utilitarias permiten definir estilos directamente en los componentes sin necesidad de hojas de estilo separadas para cada elemento.

La paleta de colores se define mediante variables CSS y clases de Tailwind. Se priorizan colores sobrios con acentos en naranja para acciones principales y gris para fondos.

El diseño responsive se implementa con las clases de Tailwind para breakpoints. El menú se colapsa en móvil, las grillas pasan a una sola columna y los formularios ocupan el ancho disponible.

Las imágenes se sirven mediante el componente Image de Next.js y el endpoint /uploads del backend. Los carteles de anuncios se almacenan en el directorio uploads del contenedor backend.

E.2. Descripción técnica del backend

El backend se implementa con Express y TypeScript. El archivo src/index.ts inicia el servidor, espera a MySQL y crea los usuarios de demostración. El archivo src/app.ts configura middlewares y rutas.

Express utiliza middlewares para procesar las peticiones en cadena. Los middlewares de seguridad se ejecutan primero, seguidos de parseo de JSON, rate limiting y autenticación. Cada ruta añade su propio middleware de validación.

La conexión a base de datos se centraliza en src/config/database.ts. Se crea un pool de conexiones con mysql2. El pool permite reutilizar conexiones y gestiona el acceso concurrente de múltiples peticiones.

Los controladores se organizan por dominio. El controlador anuncios-mysql.ts implementa la consulta con filtros dinámicos. El controlador auth.ts gestiona registro, login y tokens. El controlador admin-usuarios.ts gestiona los usuarios desde el panel admin.

Los routers se definen en src/routes. Cada router importa sus controladores y expone los endpoints. La separación entre routers y controladores mantiene el archivo de rutas limpio.

La validación de datos se realiza en dos capas. Zod valida los esquemas en el frontend. En el backend, express-validator comprueba los datos recibidos antes de pasarlos al controlador.

El manejo de errores se centraliza en src/middleware/errorHandler.ts. El middleware captura errores, registra el stack en desarrollo y devuelve mensajes seguros al cliente.

El logger utiliza Winston. Los mensajes se escriben en consola y en el archivo de logs. El middleware de Morgan registra las peticiones HTTP con detalles de método, ruta y estado.

La seguridad se refuerza con Helmet, que añade cabeceras como X-Content-Type-Options y X-Frame-Options. CORS se restringe a los orígenes definidos en la configuración.

El rate limiting limita las peticiones generales y, de forma más estricta, los endpoints de autenticación. Esto evita ataques de fuerza bruta y reduce la carga del servidor.

E.3. Descripción técnica de la base de datos

La base de datos MySQL se define en el archivo database/init.sql. El script crea la base de datos citypaj, las tablas, los índices, las claves foráneas y los datos iniciales.

El esquema utiliza el motor InnoDB para todas las tablas. InnoDB soporta transacciones, claves foráneas y bloqueos a nivel de fila, lo que mejora la integridad y concurrencia.

Las tablas de contenido comparten campos comunes como creado_at y actualizado_at, con valores por defecto current_timestamp. Esto permite ordenar y auditar los registros.

La tabla anuncios incluye índices sobre categoría, comunidad_id, provincia_id, estado_moderacion y visible. Estos índices aceleran las consultas más frecuentes del home y del panel de moderación.

La tabla anuncios asimismo utiliza un índice FULLTEXT sobre título y descripción para búsquedas por texto libre. El motor MySQL permite realizar búsquedas con MATCH AGAINST cuando el índice está configurado.

Los identificadores de usuario son UUID de 36 caracteres. Esto evita la enumeración de usuarios y facilita la generación de claves desde el backend sin depender del autoincremento.

Las claves foráneas se definen en comunidad_comentarios, propuestas_apoyos y otras tablas de relación. Algunas relaciones son lógicas, gestionadas desde los controladores mediante consultas JOIN.

El script de inserción inicial incluye datos de demostración. Los datos se generaron con nombres y textos que simulan contenido real, lo que facilita las pruebas manuales.

El volumen de Docker persiste los datos de MySQL entre reinicios. Para reiniciar desde cero se utiliza docker compose down -v, lo que elimina el volumen y fuerza la recarga de init.sql.

Las consultas del backend utilizan parámetros. Cada valor se envía separado de la sentencia SQL, lo que previene la inyección SQL sin necesidad de escapar manualmente las cadenas.

E.4. Descripción técnica de la autenticación

El flujo de autenticación comienza con el registro. El usuario introduce correo, nombre y contraseña. El backend cifra la contraseña con bcrypt y almacena el hash en la tabla usuarios.

bcrypt utiliza un factor de coste que ralentiza el cálculo del hash. En este proyecto se utiliza un factor de 10. A mayor factor, mayor seguridad, pero asimismo mayor tiempo de respuesta.

El login recibe el correo y la contraseña en texto. El backend busca el usuario, obtiene el hash almacenado y compara la contraseña con bcrypt.compare. Si coinciden, se genera un token JWT.

El token JWT incluye el id del usuario y el rol. Se firma con el secreto configurado en JWT_SECRET. El token tiene un tiempo de expiración, después del cual el frontend debe usar el token de refresco.

El token de refresco permite obtener un nuevo token de acceso sin volver a introducir la contraseña. El backend almacena o valida el refresh según la implementación.

El frontend almacena el token, generalmente en memoria o localStorage, y lo envía en la cabecera Authorization con el prefijo Bearer. El middleware de autenticación decodifica el token y añade req.user.

Las rutas protegidas consultan req.user. Si el rol no coincide con el requerido, el middleware devuelve 403. Esto asegura que solo administradores y moderadores accedan a paneles de gestión.

El cierre de sesión invalida el token en el cliente. En un entorno de producción se podría mantener una lista de tokens revocados en el servidor.

La generación de usuarios demo se realiza en src/index.ts al arrancar. Si los usuarios no existen, se crean con sus respectivos roles. Esto facilita la revisión sin necesidad de registro previo.

E.5. Descripción técnica del módulo de anuncios

El módulo de anuncios es el núcleo funcional de CityPAJ. facilita a los usuarios publicar oportunidades y a los visitantes consultarlas con filtros.

La publicación recoge los campos del formulario y opcionalmente un cartel. El frontend realiza una petición POST a /api/anuncios con el JSON de datos. El backend valida y almacena el anuncio con estado pending.

El controlador anuncios-mysql.ts construye la consulta SQL de forma dinámica. Aplica filtros de categoría, comunidad, provincia, búsqueda y paginación. La búsqueda utiliza LIKE o FULLTEXT según el campo.

El listado del home excluye los anuncios culturales. La utilidad CATEGORIAS_CULTURA define qué categorías se consideran cultura. Esta separación permite mantener una sección específica de cultura y eventos.

El detalle de anuncio muestra la información completa. Si el usuario está autenticado, puede guardar el anuncio como favorito, reportarlo o contactar con el creador.

La moderación de anuncios permite aprobar, rechazar o marcar contenido. El moderador puede añadir un motivo de rechazo. El anuncio pasa a estado approved o rejected.

La edición de anuncios permite modificar los datos del anuncio propio. El controlador verifica que el usuario que edita es el creador o un administrador.

Los anuncios se paginan en el backend. El frontend recibe el total de páginas y el listado correspondiente. La navegación de paginación se implementa con componentes de UI.

La subida de carteles se realiza con Multer. El archivo se guarda en el directorio uploads con un nombre único. La URL del cartel se almacena en el campo cartel_url de la tabla anuncios.

E.6. Descripción técnica del módulo de comunidad

El módulo de comunidad permite publicar contenido relacionado con una provincia. Las publicaciones se organizan por provincia y tema, lo que facilita la participación territorial.

Un usuario puede crear una publicación anónima. En ese caso, el sistema solicita un nombre y registra la dirección IP. La IP se utiliza únicamente para moderación.

Los comentarios están vinculados a una publicación. La tabla comunidad_comentarios almacena el contenido, el usuario, si es anónimo, la IP y el estado de moderación.

El sistema de likes permite marcar publicaciones y comentarios. La tabla comunidad_likes utiliza un índice único sobre tipo, objeto_id e IP para evitar likes duplicados.

La moderación de comunidad permite ocultar o aprobar publicaciones y comentarios. El moderador accede al panel y aplica la acción correspondiente.

El frontend muestra las publicaciones de una provincia seleccionada. El usuario puede cambiar de provincia y ver el contenido asociado.

La funcionalidad se encuentra implementada de forma parcial. La creación y consulta básica funcionan, pero la gestión avanzada de likes y reportes requiere completarse.

E.7. Descripción técnica del módulo de sugerencias y propuestas

El buzón de sugerencias recoge propuestas de mejora, quejas o ideas de los usuarios. No requiere autenticación, lo que reduce la barrera de participación.

Las sugerencias se almacenan en la tabla sugerencias con campos de asunto, descripción, categoría y prioridad. Los administradores pueden consultarlas desde el panel.

El módulo de propuestas muestra iniciativas ciudadanas. Cada propuesta incluye título, descripción y comunidad. Los usuarios pueden apoyar una propuesta.

La tabla propuestas_apoyos registra el apoyo vinculado a un usuario y una propuesta. Esto permite contar el número de apoyos de cada propuesta.

El frontend muestra las propuestas en una página. Cada propuesta indica el número de apoyos recibidos y ofrece un botón para añadir el propio.

Ambos módulos se consultan a través de la API. El backend valida los datos y devuelve el listado. La gestión desde el panel admin permite marcar sugerencias como resueltas.

E.8. Descripción técnica de cultura, eventos y recursos

La sección de cultura y eventos centraliza actividades como conciertos, talleres y encuentros. Los eventos se almacenan en la tabla eventos con fecha, provincia y descripción.

Los anuncios culturales se filtran mediante la utilidad CATEGORIAS_CULTURA. Estas categorías se excluyen del home general para mantener la sección cultural separada.

La página de cultura consulta los eventos y anuncios culturales mediante el endpoint /api/eventos y /api/anuncios?categoria=cultura. El frontend muestra tarjetas con la información.

El módulo de recursos lista convocatorias, guías y enlaces de interés. La tabla recursos almacena título, descripción, categoría y URL.

Los recursos se muestran en una página con filtros por categoría. El usuario puede pulsar un recurso para abrir la URL externa o ver el detalle.

La publicación de eventos y recursos utiliza el mismo flujo general de anuncios. La categoría distingue el tipo de contenido y determina en qué sección aparece.

E.9. Descripción técnica de administración y moderación

El panel de administración agrupa módulos de gestión. Cada módulo tiene una vista en frontend/src/app/admin y un conjunto de endpoints en /api/admin.

El módulo de usuarios permite crear, editar y desactivar cuentas. El controlador admin-usuarios.ts implementa las operaciones sobre la tabla usuarios.

El módulo de anuncios permite revisar, aprobar y rechazar anuncios en masa. El controlador admin-anuncios.ts implementa filtros y acciones masivas.

El módulo de tareas permite asignar tareas a usuarios. La tabla admin_tareas almacena título, descripción, estado, prioridad y responsable.

La agenda institucional permite anotar fechas relevantes. La tabla agenda_notas almacena título, cuerpo, fecha y color.

La mensajería interna permite redactar mensajes entre usuarios del staff. La tabla mensajes_staff almacena asunto, cuerpo, remitente y destinatario.

Las comunicaciones institucionales permiten preparar borradores de correos a entidades. Se utilizan plantillas de comunicación para rellenar asunto y cuerpo.

El panel de moderación está enfocado a reportes y contenido pendiente. El moderador puede aprobar, rechazar, ocultar o marcar publicaciones y anuncios.

Algunos paneles incluyen datos de demostración. La funcionalidad se encuentra implementada de forma parcial, con vistas funcionales pero envíos reales pendientes.

E.10. Descripción técnica de Docker

La dockerización se define en docker-compose.yml. El archivo declara tres servicios: mysql, backend y frontend. Cada servicio se construye desde su respectivo Dockerfile.

El servicio mysql utiliza la imagen oficial mysql:8.0. Se configura la base de datos citypaj, el usuario citypaj_user y la contraseña. El volumen monta database/init.sql para su ejecución inicial.

El healthcheck de mysql ejecuta una consulta `SELECT 1 FROM usuarios LIMIT 1`. Esto garantiza que la base de datos ha cargado el esquema antes de que el backend intente conectarse.

El servicio backend depende de mysql con la condición `service_healthy`. Docker Compose espera a que el contenedor mysql pase el healthcheck antes de iniciar el backend.

El backend escucha en todas las interfaces con `bind 0.0.0.0`. Esto permite que otros contenedores y el host accedan al puerto 3002.

El servicio frontend se construye con el Dockerfile del frontend. El build compila la aplicación Next.js y el comando `npm start` arranca el servidor en el puerto 3001.

La red interna de Docker permite resolver el nombre del servicio backend como `http://backend:3002`. El frontend utiliza esta URL interna para las peticiones del servidor.

Las variables de entorno se inyectan en los contenedores. El backend recibe las credenciales de MySQL, los secretos JWT y las URLs de CORS. El frontend recibe la URL pública de la API.

El reinicio de la base de datos se realiza con `docker compose down -v` seguido de `docker compose up --build`. Esto elimina el volumen y recarga el esquema y los datos iniciales.

E.11. Consideraciones de seguridad

El proyecto aplica varias medidas de seguridad básicas. La contraseña se cifra con `bcrypt`. Los tokens JWT protegen las sesiones. Las rutas se restringen por rol.

Helmet añade cabeceras HTTP que mitigan ataques comunes como clickjacking y XSS. La configuración permite inline styles para Tailwind CSS.

CORS se restringe al origen del frontend. En entorno de desarrollo se permite `http://localhost:3001`. En producción se debería configurar el dominio real.

Rate limiting limita las peticiones generales y las de autenticación. Esto reduce el riesgo de fuerza bruta y de consumo excesivo de recursos.

La validación de datos con `Zod` y `express-validator` evita el almacenamiento de datos incorrectos. asimismo ayuda a prevenir ciertos tipos de ataques de entrada.

Las consultas a MySQL utilizan parámetros. El driver mysql2 separa la sentencia SQL de los valores, lo que previene la inyección SQL.

El registro de actividad en `admin_activity_logs` permite auditar acciones relevantes. Cada entrada incluye la acción, la entidad afectada, detalles e IP del usuario.

En un despliegue real se deberían utilizar secretos fuertes, HTTPS, backups automáticos y políticas de contraseñas más estrictas. El entorno académico mantiene valores demo para facilitar la revisión.

E.12. Consideraciones de rendimiento

El rendimiento del sistema es suficiente para el entorno académico. La base de datos utiliza índices en los campos más consultados. El backend utiliza un pool de conexiones.

La consulta de anuncios aplica filtros en el backend, reduciendo la cantidad de datos transferidos. La paginación limita el número de resultados por página.

El frontend utiliza Next.js para renderizado eficiente. Los componentes se actualizan solo cuando cambian sus props o estado.

La carga de imágenes se realiza de forma asíncrona. Los carteles se sirven desde el directorio uploads, evitando la carga de la base de datos con datos binarios.

El rate limiting no solo mejora la seguridad, sino que asimismo protege el rendimiento ante picos de peticiones.

Para mejorar el rendimiento futuro se podrían añadir caché con Redis, índices adicionales y consultas optimizadas para los paneles de administración.

El contenedor de frontend se construye en la imagen. Esto aumenta el tiempo de build, pero reduce el tiempo de arranque posterior. El backend asimismo compila a dist antes de servir.

E.13. Revisión de decisiones técnicas

La decisión de utilizar MySQL frente a PostgreSQL o MongoDB se basó en el aprendizaje del ciclo. MySQL es una base de datos relacional madura y correctamente documentada.

Next.js se eligió dado que permite aprender SSR, enrutamiento moderno y componentes React en un solo proyecto. La curva de aprendizaje es mayor que la de React puro, pero ofrece más herramientas.

TypeScript se utilizó en frontend y backend. El tipado estático ayudó a detectar errores durante el desarrollo, a pesar de que asimismo añadió complejidad inicial.

Tailwind CSS permitió maquetar rápido sin escribir hojas de estilo extensas. La consistencia visual se mantuvo mediante clases compartidas.

La dockerización se añadió en definitiva del proyecto. a pesar de que requirió ajustes, facilitó la entrega al tribunal y evitó problemas de versiones.

La autenticación con JWT es adecuada para el alcance del proyecto. En un entorno real se debería añadir refresh tokens rotativos y revocación en el servidor.

La decisión de excluir anuncios culturales del home surgió al observar que la sección de cultura requería un tratamiento diferenciado. La utilidad `CATEGORIAS_CULTURA` permite mantener la separación.

E.14. Pruebas y validación

El plan de pruebas se documenta en el apartado 4.3. a pesar de que no se ejecutaron de forma automatizada, el código está preparado para pruebas manuales con los usuarios demo.

La validación de formularios se prueba introduciendo datos incorrectos. El frontend muestra mensajes de error y el backend rechaza la petición.

La autenticación se prueba con los tres usuarios demo. Cada uno debe acceder a las funciones permitidas según su rol.

El despliegue con Docker se prueba ejecutando `docker compose up --build` y accediendo a las URLs. El health check confirma que el backend conecta con MySQL.

La respuesta de la API se prueba con herramientas como el navegador, Postman o curl. Se pueden realizar peticiones GET, POST, PUT y DELETE a los endpoints.

El diseño responsive se prueba redimensionando el navegador. Se verifica que el menú, las tarjetas y los formularios se adaptan correctamente.

La carga de datos se prueba revisando que los anuncios, eventos, sugerencias y propuestas aparecen en el frontend.

E.15. Mantenimiento futuro

El mantenimiento del proyecto requiere mantener las dependencias actualizadas. npm permite actualizar paquetes con `npm update`, a pesar de que debe probarse antes de entregar.

El esquema de base de datos es posible ampliar añadiendo nuevas tablas o columnas. Cada cambio requiere reiniciar el volumen de MySQL o aplicar una migración.

Los logs del backend se almacenan en el directorio logs. Se recomienda revisarlos periódicamente para detectar errores o intentos de acceso no autorizados.

La documentación debe actualizarse si se añaden nuevos endpoints o cambian las variables de entorno. El README es el punto de entrada para nuevos revisores.

El control de versiones con Git permite revertir cambios y mantener un historial. Se recomienda usar ramas para nuevas funcionalidades.

Las copias de seguridad de la base de datos se pueden realizar exportando el volumen o el script `init.sql`. En un entorno real se debería automatizar este proceso.